

FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision

Jay Shah^{*1}, Ganesh Bikshandi^{*1}, Ying Zhang², Vijay Thakkar^{3,4}, Pradeep Ramani³, and Tri Dao^{5,6}

¹Colfax Research ²Meta ³NVIDIA ⁴Georgia Tech ⁵Princeton University ⁶Together AI
{jayhshah,ganesh}@colfax-intl.com, yingz@meta.com, {vithakkar,prraman}@nvidia.com, tri@tridao.me

July 16, 2024

Abstract

Attention, as a core layer of the ubiquitous Transformer architecture, is the bottleneck for large language models and long-context applications. FLASHATTENTION elaborated an approach to speed up attention on GPUs through minimizing memory reads/writes. However, it has yet to take advantage of new capabilities present in recent hardware, with FLASHATTENTION-2 achieving only 35% utilization on the H100 GPU. We develop three main techniques to speed up attention on Hopper GPUs: exploiting asynchrony of the Tensor Cores and TMA to (1) overlap overall computation and data movement via warp-specialization and (2) interleave block-wise matmul and softmax operations, and (3) block quantization and incoherent processing that leverages hardware support for FP8 low-precision. We demonstrate that our method, FLASHATTENTION-3, achieves speedup on H100 GPUs by 1.5-2.0× with FP16 reaching up to 740 TFLOPs/s (75% utilization), and with FP8 reaching close to 1.2 PFLOPs/s. We validate that FP8 FLASHATTENTION-3 achieves 2.6× lower numerical error than a baseline FP8 attention.

1 Introduction

For the Transformer architecture [59], the attention mechanism constitutes the primary computational bottleneck, since computing the self-attention scores of queries and keys has quadratic scaling in the sequence length. Scaling attention to longer context will unlock new capabilities (modeling and reasoning over multiple long documents [24, 43, 50] and files in large codebases [30, 48]), new modalities (high-resolution images [11], audio [23], video [25]), and new applications (user interaction with long history [53], agent workflow with long horizon [62]). This has generated significant interest in making attention faster in the long-context regime, including by approximation [14, 27, 56] and software optimization ([17, 29, 45]), or even alternative architectures [22, 42, 55].

In this work, we build on the work of Dao et al. [17] on developing exact-attention algorithms that integrate knowledge of the GPU’s execution model and hardware characteristics into their high-level design. In [17], Dao et al. introduced FLASHATTENTION, a novel tiling strategy for parallelizing attention that eliminates intermediate reads/writes to slow global memory through fusing all of the attention operations into a single GPU kernel. Dao [15] restructured the algorithm as FLASHATTENTION-2 to also parallelize over the sequence length dimension and perform the inner loop of the forward pass over blocks of the key and value matrices, thus improving the occupancy and distribution of work on the GPU. However, we observe that FLASHATTENTION-2 nonetheless achieves poor utilization on newer GPUs relative to optimized matrix-multiplication (GEMM) kernels, such as 35% vs. 80-90% on the Hopper H100 GPU. Partially, this may be attributed to implementation-level differences, such as not using Hopper-specific instructions in place of Ampere ones when targeting the Tensor Cores. Several work such as ThunkerKitten [52] and cuDNN 9 [39] has shown that with Hopper-specific instructions and tile-based abstractions, one can speedup attention computation and simplify the implementation.

^{*}Equal contribution

FlashAttention-3: 快速且准确的注意力机制：异步与低精度的结合

杰伊·沙阿^{*1}、加内什·比克尚迪^{*1}、张颖²、维贾伊·塔卡尔^{3,4}、普拉迪普·拉马尼³和特里·道^{5,6}

¹Colfax Research ²Meta ³NVIDIA ⁴Georgia Tech ⁵Princeton University ⁶Together AI
{jayhshah,ganesh}@colfax-intl.com, yingz@meta.com, {vithakkar,prraman}@nvidia.com, tri@tridao.me

2024年7月16日

摘要

注意力机制作为无处不在的Transformer架构的核心层，是大型语言模型和长上下文应用的瓶颈。FlashAttention 阐述了一种通过最小化内存读写来加速GPU上注意力计算的方法。然而该方法尚未充分利用近期硬件的新特性，FlashAttention-2在H100 GPU上仅实现了35%的利用率。我们开发了三种关键技术来加速Hopper GPU上的注意力计算：利用张量核心与TMA的异步特性实现（1）通过线程束专业化重叠整体计算与数据移动，（2）交错执行分块矩阵乘法与softmax运算，以及（3）采用支持FP8低精度计算的硬件特性进行分块量化和非相干处理。实验表明，我们的FlashAttention-3方法在H100 GPU上实现了1.5-2.0×倍加速——FP16精度下算力高达740 TFLOPs/s（75%利用率），FP8精度下算力接近1.2 PFLOPs/s。我们验证了FP8版FlashAttention-3相比基线FP8注意力计算可降低2.6×倍数值误差。

1 引言

对于Transformer架构[59]而言，注意力机制构成了主要的计算瓶颈，因为计算查询和键的自注意力分数在序列长度上具有二次缩放特性。将注意力机制扩展到更长上下文将解锁新的能力（对多篇长文档[24, 43, 50]和大型代码库中的文件[30, 48]进行建模与推理）、新模式（高分辨率图像[11]、音频[23]、视频[25]）以及新应用（用户与长历史记录[53]、长视野智能体工作流[62]）。这使得在长上下文场景下加速注意力计算引起了广泛关注，包括通过近似方法[14, 27, 56]和软件优化（[17, 29, 45]），甚至替代架构[22, 42, 55]来实现。

在这项工作中，我们基于Dao等人[17]的研究，该研究致力于开发精确注意力算法，将GPU执行模型和硬件特性的知识融入其高层设计。在[17]中，Dao等人提出了FlashAttention，这是一种新颖的并行化注意力分块策略，通过将所有注意力操作融合到单个GPU内核中，消除了对缓慢全局内存的中间读写。Dao[15]将该算法重构为FlashAttention-2，使其也能在序列长度维度上并行化，并在键值矩阵块上执行前向传播的内层循环，从而提高了GPU上的工作占用率和分配。然而，我们观察到，相对于优化的矩阵乘法（GEMM）内核，FlashAttention-2在较新的GPU上仍然利用率较低，例如在Hopper H100 GPU上为35%对比80-90%。部分原因可能归因于实现层面的差异，例如在针对Tensor Cores时未使用Hopper专用指令替代Ampere指令。ThunkerKitten[52]和cuDNN 9[39]等多项研究表明，通过使用Hopper专用指令和基于分块的抽象，可以加速注意力计算并简化实现。

^{*}Equal contribution

More fundamentally, FLASHATTENTION-2’s algorithm adheres to a simplified synchronous model and makes no explicit use of asynchrony and low-precision in its design. Asynchrony is a result of hardware specialization to accelerate the most important operations in a ML workload: specific hardware units performing matrix multiplication (Tensor Cores) or memory loading (Tensor Memory Accelerator – TMA), separate from the rest of the CUDA cores performing logic, integer, and floating point computation. Low precision such as FP8 in Hopper and FP4 in Blackwell, continuing the trend of FP16 (Pascal in 2017) and BF16 (Ampere in 2020), is a proven technique to get double or quadruple throughput for the same power and chip area. We review the capabilities afforded by Hopper in these directions in §2.2. The technical challenge is to redesign FLASHATTENTION-2 to make use of these hardware features: asynchrony requires overlapping computation between matmul and softmax even though one depends on the output of the other, and low-precision requires care to minimize quantization error, especially in the case of outlier features in LLMs [20, 54].

To this end, we propose FLASHATTENTION-3, which contributes and synthesizes three new ideas to further improve performance on newer GPU architectures:¹

1. **Producer-Consumer asynchrony:** We define a warp-specialized software pipelining scheme that exploits the asynchronous execution of data movement and Tensor Cores by splitting producers and consumers of data into separate warps, thereby extending the algorithm’s ability to hide memory and instruction issue latencies.
2. **Hiding softmax under asynchronous block-wise GEMMs:** We overlap the comparatively low-throughput non-GEMM operations involved in softmax, such as floating point multiply-add and exponential, with the asynchronous WGMMMA instructions for GEMM. As part of this, we rework the FLASHATTENTION-2 algorithm to circumvent certain sequential dependencies between softmax and the GEMMs. For example, in the 2-stage version of our algorithm, while softmax executes on one block of the scores matrix, WGMMMA executes in the asynchronous proxy to compute the next block.
3. **Hardware-accelerated low-precision GEMM:** We adapt the forward pass algorithm to allow for targeting the FP8 Tensor Cores for GEMM, nearly doubling the measured TFLOPs/s. This requires bridging the different layout conformance requirements of WGMMMA in terms of how blocks of FP32 accumulator and FP8 operand matrices are assumed to be laid out in memory. We use the techniques of block quantization and incoherent processing to mitigate the loss of accuracy that results from moving to FP8 precision.

To validate our method empirically, we benchmark FLASHATTENTION-3 on the H100 SXM5 GPU over a range of parameters and show that (1) FP16 achieves 1.5-2.0× speedup over FLASHATTENTION-2 in the forward pass (reaching up to 740 TFLOPs/s) and 1.5-1.75× in the backward pass, (2) FP8 achieves close to 1.2 PFLOPs/s, and (3) for large sequence length, FP16 outperforms and FP8 is competitive² with a state-of-the-art implementation of attention from NVIDIA’s cuDNN library. We also validate that FP16 FLASHATTENTION-3 yields the same numerical error as FLASHATTENTION-2 and is better than the standard attention implementation as intermediate results (e.g., softmax rescaling) are kept in FP32. Moreover, FP8 FLASHATTENTION-3 with block quantization and incoherent processing is 2.6× more accurate than standard attention with per-tensor quantization in cases with outlier features.

We open-source FLASHATTENTION-3 with a permissive license³ and plan to integrate it with PyTorch and Hugging Face libraries to benefit the largest number of researchers and developers.

2 Background: Multi-Head Attention and GPU Characteristics

2.1 Multi-Head Attention

Let $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ be the query, key and value input sequences associated to a single head, where N is the sequence length and d is the head dimension. Then the attention output $\mathbf{O}$ is computed as:

$$\mathbf{S} = \alpha \mathbf{QK}^\top \in \mathbb{R}^{N \times N}, \quad \mathbf{P} = \text{softmax}(\mathbf{S}) \in \mathbb{R}^{N \times N}, \quad \mathbf{O} = \mathbf{PV} \in \mathbb{R}^{N \times d},$$

¹We describe our results in the context of NVIDIA’s Hopper architecture. However, our algorithm is operative for any GPU architecture with sufficiently robust asynchronous execution and low-precision capabilities.

²More precisely, for head dimension 64 FLASHATTENTION-3 FP8 is ahead, while for head dimensions 128 and 256 it is at par for those cases without causal masking and behind with causal masking.

³FLASHATTENTION-3 is available at <https://github.com/Dao-AILab/flash-attention>

更根本的是，FlashAttention-2的算法遵循简化的同步模型，其设计中没有明确使用异步性和低精度。异步性是硬件专门化的结果，旨在加速机器学习工作负载中最重要的操作：特定的硬件单元执行矩阵乘法（张量核心）或内存加载（张量内存加速器——TMA），与执行逻辑、整数和浮点计算的其余CUDA核心分离。低精度（如Hopper中的FP8和Blackwell中的FP4，延续了FP16（2017年Pascal架构）和BF16（2020年Ampere架构）的趋势）是一种经过验证的技术，可在相同功耗和芯片面积下实现双倍或四倍的吞吐量。我们在§2.2中回顾了Hopper在这些方面提供的的能力。技术挑战在于重新设计FlashAttention-2以利用这些硬件特性：异步性要求即使矩阵乘法和softmax之间存在依赖关系，也要实现两者计算的重叠；而低精度则需要谨慎处理以最小化量化误差，尤其是在大语言模型中存在异常特征的情况下[20, 54]。

为此，我们提出了FlashAttention-3，它贡献并综合了三个新思路，以进一步提升在新型GPU架构上的性能：¹

1. 生产者-消费者异步机制：我们定义了一种经warp特化的软件流水线方案，该方案通过将数据生产者与消费者分离至不同的warp中，利用数据移动与Tensor Core的异步执行特性，从而扩展算法隐藏内存及指令发射延迟的能力。
2. 在异步分块GEMM下隐藏softmax：我们将softmax中涉及的低吞吐量非GEMM操作（如浮点乘加运算与指数运算）与用于GEMM的异步WGMMMA指令进行重叠执行。为此，我们重构了FlashAttention-2算法以规避softmax与GEMM间的部分顺序依赖。例如，在我们算法的两阶段版本中，当softmax在分数矩阵的某个分块上执行时，WGMMMA同时在异步代理中执行以计算下一个分块。
3. 硬件加速低精度GEMM：我们调整前向传播算法以适配FP8 Tensor Core执行GEMM运算，使实测TFLOPs/s提升近一倍。这需要弥合WGMMMA在FP32累加器矩阵与FP8操作数矩阵分块的内存布局假设方面的差异。我们采用分块量化和非相干处理技术来缓解转向FP8精度导致的精度损失。

为了通过实验验证我们的方法，我们在H100 SXM5 GPU上对FlashAttention-3进行了一系列参数基准测试，结果表明：(1) 在前向传播中，FP16相比FlashAttention-2实现了1.5-2.0×的加速（最高达到740 TFLOPs/s），在反向传播中实现了1.5-1.75×的加速；(2) FP8实现了接近1.2 PFLOPs/s的性能；(3) 对于长序列，FP16表现更优，而FP8与NVIDIA cuDNN库中最先进的注意力实现相比具有竞争力²。我们还验证了FP16 FlashAttention-3产生的数值误差与FlashAttention-2相同，并且由于中间结果（例如softmax重缩放）保持在FP32精度，其表现优于标准注意力实现。此外，在存在异常特征的情况下，采用块量化和非相干处理的FP8 FlashAttention-3比采用逐张量量化的标准注意力实现准确度高出2.6×倍。

我们以宽松许可证开源FlashAttention-3³，并计划将其集成到PyTorch中，以方便研究人员和开发者使用。Hugging Face 库旨在惠及最广泛的研究人员和开发者群体。

2 背景：多头注意力机制与GPU特性

2.1 多头注意力机制

设 $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ 为与单个注意力头相关联的查询、键和值输入序列，其中 N 为序列长度， d 为注意力头维度。则注意力输出 $\mathbf{O}$ 的计算方式如下：

$$\mathbf{S} = \alpha \mathbf{QK}^\top \in \mathbb{R}^{N \times N}, \quad \mathbf{P} = \text{softmax}(\mathbf{S}) \in \mathbb{R}^{N \times N}, \quad \mathbf{O} = \mathbf{PV} \in \mathbb{R}^{N \times d},$$

¹We describe our results in the context of NVIDIA’s Hopper architecture. However, our algorithm is operative for any GPU architecture with sufficiently robust asynchronous execution and low-precision capabilities.

²More precisely, for head dimension 64 FLASHATTENTION-3 FP8 is ahead, while for head dimensions 128 and 256 it is at par for those cases without causal masking and behind with causal masking.

³FLASHATTENTION-3 is available at <https://github.com/Dao-AILab/flash-attention>

where softmax is applied row-wise and one typically sets $\alpha = 1/\sqrt{d}$ as the scaling factor. In practice, we subtract $\text{rowmax}(\mathbf{S})$ from $\mathbf{S}$ to prevent numerical instability with the exponential function. For multi-head attention (MHA), each head has its own set of query, key and value projections, and this computation parallelizes across multiple heads and batches to produce the full output tensor.

Now let ϕ be a scalar loss function and let $\mathbf{d}(-) = \partial\phi/\partial(-)$ be notation for the gradient. Given the output gradient $\mathbf{dO} \in \mathbb{R}^{N \times d}$, we compute $\mathbf{dQ}$, $\mathbf{dK}$, and $\mathbf{dV}$ according to the chain rule as follows:

$$\begin{aligned}\mathbf{dV} &= \mathbf{P}^\top \mathbf{dO} \in \mathbb{R}^{N \times d} \\ \mathbf{dP} &= \mathbf{dOV}^\top \in \mathbb{R}^{N \times N} \\ \mathbf{dS} &= \text{dsoftmax}(\mathbf{dP}) \in \mathbb{R}^{N \times N} \\ \mathbf{dQ} &= \alpha \mathbf{dSK} \in \mathbb{R}^{N \times d} \\ \mathbf{dK} &= \alpha \mathbf{dS}^\top \mathbf{Q} \in \mathbb{R}^{N \times d},\end{aligned}$$

Here, we have that $\mathbf{ds} = (\text{diag}(p) - pp^\top)\mathbf{dp}$ for $p = \text{softmax}(s)$ as a function of a vector s , and we write $\text{dsoftmax}(\mathbf{dP})$ for this formula applied row-wise. Finally, this computation again parallelizes across the number of heads and batches for the backward pass of MHA.

2.2 GPU hardware characteristics and execution model

We describe the aspects of the GPU’s execution model relevant for FLASHATTENTION-3, with a focus on the NVIDIA Hopper architecture as a concrete instantiation of this model.

Memory hierarchy: The GPU’s memories are organized as a hierarchy of data locales, with capacity inversely related to bandwidth (Table 1)⁴. Global memory (GMEM), also known as HBM, is the off-chip DRAM accessible to all streaming multiprocessors (SMs). Data from GMEM gets transparently cached into an on-chip L2 cache. Next, each SM contains a small on-chip, programmer-managed highly banked cache called shared memory (SMEM). Lastly, there is the register file within each SM.

Thread hierarchy: The GPU’s programming model is organized around logical groupings of execution units called threads. From the finest to coarsest level, the thread hierarchy is comprised of threads, warps (32 threads), warpgroups (4 contiguous warps), threadblocks (i.e., cooperative thread arrays or CTAs), threadblock clusters (in Hopper), and grids.

These two hierarchies are closely interlinked. Threads in the same CTA are co-scheduled on the same SM, and CTAs in the same cluster are co-scheduled on the same GPC. SMEM is directly addressable by all threads within a CTA, whereas each thread has at most 256 registers (RMEM) private to itself.

Table 1: Thread-Memory hierarchy for the NVIDIA Hopper H100 SXM5 GPU.

Hardware Level	Parallel Agent	Data Locale	Capacity @ Bandwidth
Chip	Grid	GMEM	80 GiB @ 3.35 TB/s
GPC	Threadblock Clusters	L2	50 MiB @ 12 TB/s
SM	Threadblock (CTA)	SMEM	228 KiB per SM, 31TB/s per GPU
Thread	Thread	RMEM	256 KiB per SM

Asynchrony and warp-specialization: GPUs are throughput processors that rely on concurrency and asynchrony to hide memory and execution latencies. For async memory copy between GMEM and SMEM, Hopper has the Tensor Memory Accelerator (TMA) as a dedicated hardware unit [38, §7.29]. Furthermore, unlike prior architectures such as Ampere, the Tensor Core of Hopper, exposed via the warpgroup-wide WGMMMA instruction [40, §9.7.14], is also asynchronous and can source its inputs directly from shared memory.

⁴Luo et al. [34] reports shared memory bandwidth of 128 bytes per clock cycle per SM, and we multiply that by 132 SMs and the boost clock of 1830 MHz.

其中softmax按行应用，通常将缩放因子 设为 $1/\sqrt{}$ 。在实践中，我们会从输入中减去行最大值以防止指数函数带来的数值不稳定问题。对于多头注意力机制（MHA），每个头都有自己独立的查询、键和值投影，该计算过程会在多个头和批次间并行执行，最终生成完整的输出张量。

现在令 ℓ 为一个标量损失函数，并令 $\mathbf{d}(-) = \partial \ell / \partial (-)$ 表示梯度符号。给定输出梯度 $\mathbf{dO} \in \mathbb{R}^{N \times d}$ ，我们根据链式法则计算 $\mathbf{dQ}$ 、 $\mathbf{dK}$ 和 $\mathbf{dV}$ ，如下所示：

$$\begin{aligned}\mathbf{dV} &= \mathbf{P}^\top \mathbf{dO} \in \mathbb{R}^{N \times d} \\ \mathbf{dP} &= \mathbf{dOV}^\top \in \mathbb{R}^{N \times N} \\ \mathbf{dS} &= \text{dsoftmax}(\mathbf{dP}) \in \mathbb{R}^{N \times N} \\ \mathbf{dQ} &= \alpha \mathbf{dSK} \in \mathbb{R}^{N \times d} \\ \mathbf{dK} &= \alpha \mathbf{dS}^\top \mathbf{Q} \in \mathbb{R}^{N \times d},\end{aligned}$$

这里，我们有 $\mathbf{d} = (\text{diag}(\mathbf{p}) - \mathbf{p}\mathbf{p}^\top)\mathbf{d}$ 对于 $\mathbf{p} = \text{softmax}(\mathbf{s})$ 作为向量 $\mathbf{s}$ 的函数，并且我们将逐行应用此公式记为 $\text{dsoftmax}(\mathbf{dP})$ 。最后，对于多头注意力机制的反向传播，该计算再次在头数和批次维度上并行化。

2.2 GPU硬件特性与执行模型

我们描述了GPU执行模型中与FlashAttention-3相关的方面，重点在于 NVIDIA Hopper架构作为该模型的一个具体实例。

内存层次结构：GPU的内存被组织为数据位置的层次结构，其容量与带宽成反比（表1）⁴。全局内存（GMEM），也称为HBM，是所有流式多处理器（SM）均可访问的片外DRAM。来自GMEM的数据会被透明地缓存到片上的L2缓存中。接着，每个SM包含一个小的片上、由程序员管理的高度分组的缓存，称为共享内存（SMEM）。最后，每个SM内部还有寄存器文件。

线程层次结构：GPU的编程模型围绕称为线程的执行单元的逻辑分组进行组织。从最精细到最粗略的层级，线程层次结构由线程、线程束（32个线程）、线程束组（4个连续线程束）、线程块（即协作线程数组或CTA）、线程块簇（在Hopper架构中）和网格组成。

这两个层次结构紧密相连。同一CTA内的线程在同一SM上协同调度，而同一集群内的CTA在同一GPC上协同调度。SMEM可由CTA内的所有线程直接寻址，而每个线程最多拥有256个仅供自身使用的寄存器（RMEM）。

表1：Thread-Memory hierarchy for the NVIDIA Hopper H100 SXM5 GPU.

Hardware Level	Parallel Agent	Data Locale	Capacity @ Bandwidth
Chip	Grid	GMEM	80 GiB @ 3.35 TB/s
GPC	Threadblock Clusters	L2	50 MiB @ 12 TB/s
SM	Threadblock (CTA)	SMEM	228 KiB per SM, 31TB/s per GPU
Thread	Thread	RMEM	256 KiB per SM

异步性与线程束专业化：GPU是吞吐型处理器，依赖并发性和异步性来隐藏内存与执行延迟。对于全局内存与共享内存间的异步内存复制，Hopper架构配备了作为专用硬件单元的Tensor Memory Accelerator（TMA）[38, §7.29]。此外，与Ampere等前代架构不同，Hopper的Tensor Core通过线程束组范围的WGMMMA指令[40, §9.7.14]实现，同样具备异步特性，并能直接从共享内存获取输入数据。

⁴Luo等人[34]报告了每个SM每时钟周期128字节的共享内存带宽，我们将其乘以132个SM和1830 MHz的加速时钟频率。

Hardware support for asynchrony allows for warp-specialized kernels, where the warps of a CTA are divided into producer or consumer roles that only ever issue either data movement or computation. Generically, this improves the compiler’s ability to generate optimal instruction schedules [4]. In addition, Hopper supports the dynamic reallocation of registers between warpgroups via `setmaxnreg` [40, §9.7.17.1], so those warps doing MMAs can obtain a larger share of RMEM than those just issuing TMA (for which only a single thread is needed).

Low-precision number formats: Modern GPUs have specialized hardware units for accelerating low-precision computation. For example, the WGMMMA instruction can target the FP8 Tensor Cores on Hopper to deliver 2x the throughput per SM when compared to FP16 or BF16.

However, correctly invoking FP8 WGMMMA entails understanding the layout constraints on its operands. Given a GEMM call to multiply $A \times B^T$ for an $M \times K$ -matrix A and an $N \times K$ -matrix B , we say that the A or B operand is *mn-major* if it is contiguous in the outer M or N dimension, and *k-major* if it is instead contiguous in the inner K -dimension. Then for FP16 WGMMMA, both mn-major and k-major input operands are accepted for operands in SMEM, but for FP8 WGMMMA, only the k-major format is supported. Moreover, in situations such as attention where one wants to fuse back-to-back GEMMs in a single kernel, clashing FP32 accumulator and FP8 operand layouts pose an obstacle to invoking dependent FP8 WGMMAs.

In the context of attention, these layout restrictions entail certain modifications to the design of an FP8 algorithm, which we describe in §3.3.

2.3 Standard Attention and Flash Attention

Following Dao et al. [17], we let **standard attention** denote an implementation of attention on the GPU that materializes the intermediate matrices **S** and **P** to HBM. The main idea of FLASHATTENTION was to leverage a local version of the softmax reduction to avoid these expensive intermediate reads/writes and fuse attention into a single kernel. Local softmax corresponds to lines 18-19 of the consumer mainloop in Algorithm 1 together with the rescalings of blocks of **O**. The simple derivation that this procedure indeed computes **O** can be found in [15, §2.3.1].

3 FlashAttention-3: Algorithm

In this section, we describe the FLASHATTENTION-3 algorithm. For simplicity, we focus on the forward pass, with the backward pass algorithm described in Appendix B.1. We first indicate how to integrate warp-specialization with a circular SMEM buffer into the base algorithm of FLASHATTENTION-2. We then explain how to exploit asynchrony of WGMMMA to define an overlapped GEMM-softmax 2-stage pipeline. Finally, we describe the modifications needed for FP8, both in terms of layout conformance and accuracy via block quantization and incoherent processing.

3.1 Producer-Consumer asynchrony through warp-specialization and pingpong scheduling

Warp-specialization As with FLASHATTENTION-2, the forward pass of FLASHATTENTION-3 is embarrassingly parallel in the batch size, number of heads, and query sequence length. Thus, it will suffice to give a CTA-level view of the algorithm, which operates on a tile $\mathbf{Q}_i$ of the query matrix to compute the corresponding tile $\mathbf{O}_i$ of the output. To simplify the description, we first give the warp-specialization scheme with a circular SMEM buffer that does *not* have in addition the GEMM-softmax overlapping. Let d be the head dimension, N the sequence length, and fix a query block size B_r to divide $\mathbf{Q}$ into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$.

对异步的硬件支持允许实现warp专用内核，其中CTA的warp被划分为生产者或消费者角色，这些角色仅执行数据移动或计算。通常，这提高了编译器生成最优指令调度[4]的能力。此外，Hopper通过setmaxnreg支持在warp组之间动态重新分配寄存器[40, §9.7.17.1]，因此执行MMA的warp可以获得比仅执行TMA（仅需单个线程）的warp更大份额的RMEM。

低精度数字格式：现代GPU配备了专门的硬件单元，用于加速低精度计算。例如，WGMMMA指令可以针对Hopper架构上的FP8 Tensor Cores，实现相比FP16或BF16每SM吞吐量翻倍的效果。

然而，正确调用FP8 WGMMMA需要理解其操作数的布局约束。给定一个GEMM调用，用于将 $\mathbf{X}$ 与 $\mathbf{Y}^T$ 相乘，其中 $\mathbf{X}$ 是一个 $m \times k$ 矩阵， $\mathbf{Y}$ 是一个 $k \times n$ 矩阵，如果 $\mathbf{X}$ 或 $\mathbf{Y}$ 操作数在外部 m 或 n 维度上是连续的，我们称其为mn主序；如果它在内部 k 维度上是连续的，则称为k主序。那么对于FP16 WGMMMA，SMEM中的操作数既接受mn主序也接受k主序输入操作数，但对于FP8 WGMMMA，仅支持k主序格式。此外，在诸如注意力机制等场景中，若希望在单个内核中融合背靠背的GEMM，冲突的FP32累加器和FP8操作数布局会对调用依赖的FP8 WGMMMA构成障碍。

在注意力机制的背景下，这些布局限制需要对FP8算法的设计进行某些修改，我们将在§3.3中描述。

2.3 标准注意力与闪电注意力

继Dao等人[17]之后，我们让标准注意力表示在GPU上实现注意力时，将中间矩阵**S**和**P**物化到HBM中。FlashAttention的主要思想是利用局部版本的softmax归约来避免这些昂贵的中间读写操作，并将注意力融合到单个内核中。局部softmax对应于算法1中消费者主循环的第18-19行以及**O**块的重缩放。这一过程确实计算**O**的简单推导可在[15, §2.3.1]中找到。

3 FlashAttention-3：算法

在本节中，我们介绍FlashAttention-3算法。为简化说明，我们重点讨论前向传播过程，反向传播算法详见附录B.1。我们首先说明如何将warp-specialization与循环SMEM缓冲区整合到FlashAttention-2的基础算法中。接着解释如何利用WGMMMA的异步特性，构建重叠的GEMM-softmax两阶段流水线。最后，我们将阐述FP8所需的修改，包括布局一致性以及通过块量化和非相干处理实现的精度优化。

3.1 通过warp专用化和乒乓调度实现生产者-消费者异步

Warp-specialization 与FlashAttention-2一样，FlashAttention-3的前向传播在批次大小、注意力头数量和查询序列长度方面具有极高的并行性。因此，给出算法在CTA级别的视图就足够了，该算法对查询矩阵的一个分块 $\mathbf{Q}_i$ 进行操作，以计算输出对应的分块 $\mathbf{O}_i$ 。为了简化描述，我们首先给出使用循环SMEM缓冲区的warp-specialization方案，该方案不包含额外的GEMM-softmax重叠。设 d 为头维度， N 为序列长度，并固定查询块大小 B_r ，将 $\mathbf{Q}$ 分成 $T_r = \lceil \frac{N}{B_r} \rceil$ 个块 $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ 。

Algorithm 1 FLASHATTENTION-3 forward pass **without** intra-consumer overlapping – CTA view

Require: Matrices $\mathbf{Q}_i \in \mathbb{R}^{B_r \times d}$ and $\mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, key block size B_c with $T_c = \lceil \frac{N}{B_c} \rceil$.

- 1: Initialize pipeline object to manage barrier synchronization with s -stage circular SMEM buffer.
- 2: **if** in producer warpgroup **then**
- 3: Deallocate predetermined number of registers.
- 4: Issue load $\mathbf{Q}_i$ from HBM to shared memory.
- 5: Upon completion, commit to notify consumer of the load of $\mathbf{Q}_i$.
- 6: **for** $0 \leq j < T_c$ **do**
- 7: Wait for the $(j \% s)$ th stage of the buffer to be consumed.
- 8: Issue loads of $\mathbf{K}_j, \mathbf{V}_j$ from HBM to shared memory at the $(j \% s)$ th stage of the buffer.
- 9: Upon completion, commit to notify consumers of the loads of $\mathbf{K}_j, \mathbf{V}_j$.
- 10: **end for**
- 11: **else**
- 12: Reallocate predetermined number of registers as function of number of consumer warps.
- 13: On-chip, initialize $\mathbf{O}_i = (0) \in \mathbb{R}^{B_r \times d}$ and $\ell_i, m_i = (0), (-\infty) \in \mathbb{R}^{B_r}$.
- 14: Wait for $\mathbf{Q}_i$ to be loaded in shared memory.
- 15: **for** $0 \leq j < T_c$ **do**
- 16: Wait for $\mathbf{K}_j$ to be loaded in shared memory.
- 17: Compute $\mathbf{S}_i^{(j)} = \mathbf{Q}_i \mathbf{K}_j^T$ (SS-GEMM). Commit and wait.
- 18: Store $m_i^{\text{old}} = m_i$ and compute $m_i = \max(m_i^{\text{old}}, \text{rowmax}(\mathbf{S}_i^{(j)}))$.
- 19: Compute $\tilde{\mathbf{P}}_i^{(j)} = \exp(\mathbf{S}_i^{(j)} - m_i)$ and $\ell_i = \exp(m_i^{\text{old}} - m_i)\ell_i + \text{rowsum}(\tilde{\mathbf{P}}_i^{(j)})$.
- 20: Wait for $\mathbf{V}_j$ to be loaded in shared memory.
- 21: Compute $\mathbf{O}_i = \text{diag}(\exp(m_i^{\text{old}} - m_i))^{-1} \mathbf{O}_i + \tilde{\mathbf{P}}_i^{(j)} \mathbf{V}_j$ (RS-GEMM). Commit and wait.
- 22: Release the $(j \% s)$ th stage of the buffer for the producer.
- 23: **end for**
- 24: Compute $\mathbf{O}_i = \text{diag}(\ell_i)^{-1} \mathbf{O}_i$ and $L_i = m_i + \log(\ell_i)$.
- 25: Write $\mathbf{O}_i$ and L_i to HBM as the i th block of $\mathbf{O}$ and L .
- 26: **end if**

For our implementation of Algorithm 1 on Hopper, we use **setmaxnreg** for (de)allocations, TMA for loads of $\mathbf{Q}_i$ and $\{\mathbf{K}_j, \mathbf{V}_j\}_{0 \leq j < T_c}$, and WGMMMA to execute the GEMMs in the consumer mainloop, with the SS or RS prefix indicating whether the first operand is sourced from shared memory or register file. For interpreting the execution flow of Algorithm 1, note that issuing TMA loads does not stall on the completion of other loads due to asynchrony. Moreover, in the producer mainloop, no waits will be issued for the first s iterations as the buffer gets filled.

Pingpong scheduling The asynchronous nature of WGMMMA and TMA, along with warp-specialization, opens up the opportunity to overlap the softmax computation of one warpgroup with the GEMM of another warpgroup. To motivate this, notice that non-matmul operations have much lower throughput than matmul operations on modern hardware accelerators. As an example, the H100 SXM5 GPU has 989 TFLOPS of FP16 matmul but only 3.9 TFLOPS of special functions such as exponential⁵ (necessary for softmax). For the attention forward pass in FP16 with head dimension 128, there are 512x more matmul FLOPS compared to exponential operations, but the exponential has 256x lower throughput, so exponential can take 50% of the cycle compared to matmul. The situation is even worse with FP8, where the matmul throughput doubles but the exponential throughput stays the same.

Since the exponential is performed by a separate hardware unit (the multi-function unit), ideally we’d want the exponential calculation to be scheduled when the Tensor Cores are performing the matmul. To do so, we use synchronization barriers (**bar.sync** instructions) to force the GEMMs (GEMM1 – $\mathbf{P}\mathbf{V}$ of one iteration, and GEMM0 – $\mathbf{Q}\mathbf{K}^T$ of the next iteration) of warpgroup 1 to be scheduled before the GEMMs of warpgroup 2. As a result, the softmax of warpgroup 1 will be scheduled while warpgroup 2 is performing its GEMMs. Then the roles swap, with warpgroup 2 doing softmax while warpgroup 1 doing GEMMs (hence, “pingpong” scheduling). This is illustrated

⁵The CUDA programming guide specifies that 16 operations of special functions can be performed per streaming multiprocessor (SM) per clock cycle. We multiply 16 by 132 SMs and 1830 MHz clock speed to get 3.9 TFLOPS of special functions.

Algorithm 1 FLASHATTENTION-3 forward pass **without** intra-consumer overlapping – CTA view

Require: Matrices $\mathbf{Q}_i \in \mathbb{R}^{B_r \times d}$ and $\mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, key block size B_c with $T_c = \lceil \frac{N}{B_c} \rceil$.

- 1: Initialize pipeline object to manage barrier synchronization with s -stage circular SMEM buffer.
- 2: **if** in producer warpgroup **then**
- 3: Deallocate predetermined number of registers.
- 4: Issue load $\mathbf{Q}_i$ from HBM to shared memory.
- 5: Upon completion, commit to notify consumer of the load of $\mathbf{Q}_i$.
- 6: **for** $0 \leq j < T_c$ **do**
- 7: Wait for the $(j \% s)$ th stage of the buffer to be consumed.
- 8: Issue loads of $\mathbf{K}_j, \mathbf{V}_j$ from HBM to shared memory at the $(j \% s)$ th stage of the buffer.
- 9: Upon completion, commit to notify consumers of the loads of $\mathbf{K}_j, \mathbf{V}_j$.
- 10: **end for**
- 11: **else**
- 12: Reallocate predetermined number of registers as function of number of consumer warps.
- 13: On-chip, initialize $\mathbf{O}_i = (0) \in \mathbb{R}^{B_r \times d}$ and $\ell_i, m_i = (0), (-\infty) \in \mathbb{R}^{B_r}$.
- 14: Wait for $\mathbf{Q}_i$ to be loaded in shared memory.
- 15: **for** $0 \leq j < T_c$ **do**
- 16: Wait for $\mathbf{K}_j$ to be loaded in shared memory.
- 17: Compute $\mathbf{S}_i^{(j)} = \mathbf{Q}_i \mathbf{K}_j^T$ (SS-GEMM). Commit and wait.
- 18: Store $m_i^{\text{old}} = m_i$ and compute $m_i = \max(m_i^{\text{old}}, \text{rowmax}(\mathbf{S}_i^{(j)}))$.
- 19: Compute $\tilde{\mathbf{P}}_i^{(j)} = \exp(\mathbf{S}_i^{(j)} - m_i)$ and $\ell_i = \exp(m_i^{\text{old}} - m_i)\ell_i + \text{rowsum}(\tilde{\mathbf{P}}_i^{(j)})$.
- 20: Wait for $\mathbf{V}_j$ to be loaded in shared memory.
- 21: Compute $\mathbf{O}_i = \text{diag}(\exp(m_i^{\text{old}} - m_i))^{-1} \mathbf{O}_i + \tilde{\mathbf{P}}_i^{(j)} \mathbf{V}_j$ (RS-GEMM). Commit and wait.
- 22: Release the $(j \% s)$ th stage of the buffer for the producer.
- 23: **end for**
- 24: Compute $\mathbf{O}_i = \text{diag}(\ell_i)^{-1} \mathbf{O}_i$ and $L_i = m_i + \log(\ell_i)$.
- 25: Write $\mathbf{O}_i$ and L_i to HBM as the i th block of $\mathbf{O}$ and L .
- 26: **end if**

在我们于Hopper上实现算法1时，对于（解）分配操作使用setmaxnreg，对于 $\mathbf{Q}_i$ 和 $\{\mathbf{K}_j, \mathbf{V}_j\}_{0 \leq j < T_c}$ 的加载使用TMA，并在消费者主循环中使用WGMMMA执行GEMM运算，其中SS或RS前缀指示第一个操作数来自共享内存还是寄存器文件。在解读算法1的执行流程时，请注意由于异步性，发出TMA加载不会因其他加载的完成而停滞。此外，在生产者主循环中，前 次迭代不会发出等待指令，因为缓冲区正在被填充。

乒乓调度 WGMMMA和TMA的异步特性，结合线程束专业化，为重叠一个线程束组的softmax计算与另一个线程束组的GEMM计算创造了机会。为了说明这一点，请注意在现代硬件加速器上，非矩阵乘法运算的吞吐量远低于矩阵乘法运算。例如，H100 SXM5 GPU的FP16矩阵乘法算力为989 TFLOPS，而softmax)所需的指数⁵（等特殊函数算力仅为3.9 TFLOPS。对于头维度为128的FP16注意力前向传播，矩阵乘法的FLOPS数量是指数运算的512倍，但指数运算的吞吐量却低256倍，因此指数运算可能占用相当于矩阵乘法50%的周期时间。在FP8情况下问题更为严重：矩阵乘法吞吐量翻倍，而指数运算吞吐量保持不变。

由于指数运算由独立的硬件单元（多功能单元）执行，理想情况下我们希望指数计算能在张量核心执行矩阵乘法时被调度。为此，我们使用同步屏障（bar.sync指令）强制将warpgroup 1的GEMM运算（一次迭代的GEMM1 – $\mathbf{P}\mathbf{V}$ 与下一次迭代的GEMM0 – $\mathbf{Q}\mathbf{K}^T$ ）调度在warpgroup 2的GEMM运算之前。这样，当warpgroup 2执行其GEMM运算时，warpgroup 1的softmax运算将被调度。随后两者角色互换：warpgroup 2执行softmax时warpgroup 1执行GEMM运算（即形成“乒乓”调度模式）。该过程如图所示

⁵The CUDA programming guide specifies that 16 operations of special functions can be performed per streaming multiprocessor (SM) per clock cycle. We multiply 16 by 132 SMs and 1830 MHz clock speed to get 3.9 TFLOPS of special functions.

in Fig. 1. Though in practice the pingpong scheduling is not as clean as depicted in the figure, we generally find this to improve performance (e.g., from 570 TFLOPS to 620-640 TFLOPS for FP16 forward with head dimension 128 and sequence length 8192).

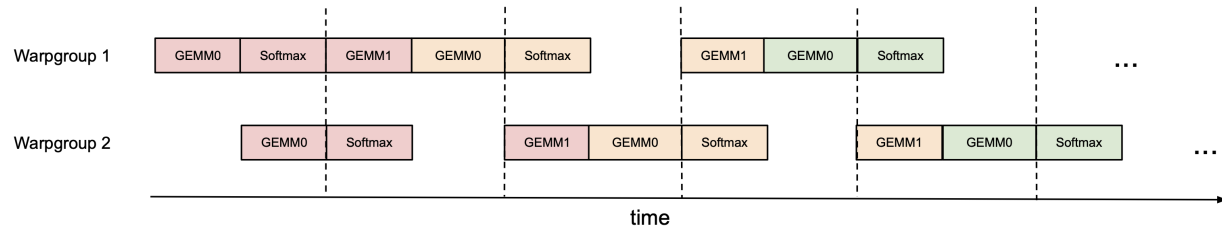


Figure 1: Pingpong scheduling for 2 warpgroups to overlap softmax and GEMMs: the softmax of one warpgroup should be scheduled when the GEMMs of another warpgroup are running. The same color denotes the same iteration.

Attention variants For multi-query attention [51] and grouped query attention [3], we follow the approach in FLASHATTENTION-2 and adjust the tensor indexing to avoid duplicating $\mathbf{K}$ and $\mathbf{V}$ in HBM.

3.2 Intra-warpgroup overlapping GEMMs and softmax

Even within one warpgroup, we can overlap some instructions in the softmax with some instructions in the GEMMs. We describe one technique to do so.

In the attention algorithm, operations within the inner loop (main loop) have sequential dependencies that impede parallelization within a single iteration. For example, (local) softmax (lines 18 to 19) relies on the output $\mathbf{S}_i^{(j)}$ of the first GEMM, while the second GEMM takes its result $\tilde{\mathbf{P}}_i^{(j)}$ as an operand. Indeed, the wait statements in lines 17 and 21 of Algorithm 1 serialize the execution of softmax and GEMMs. However, we can break these dependencies by pipelining across iterations through additional buffers in registers. Pursuing this idea, we propose the following two-stage⁶ GEMM-softmax pipelining algorithm:

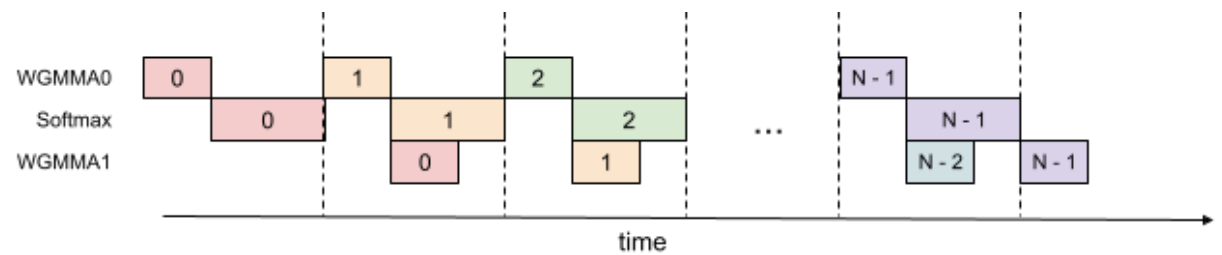


Figure 2: 2-stage WGMMMA-softmax pipelining

⁶Note that the number of stages of the overlapping scheme is bounded by, but need not equal, the number s of stages in the circular SMEM buffer.

在图1中。尽管在实际应用中，乒乓调度并不像图中描绘的那样简洁，但我们通常发现这种做法能提升性能（例如，对于头维度为128、序列长度为8192的FP16前向计算，性能从570 TFLOPS提升至620-640 TFLOPS）。

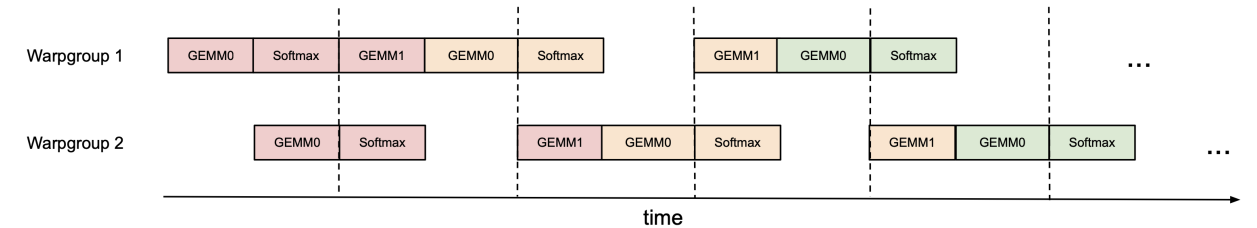


图1: 两个warpgroup的乒乓调度以重叠softmax和GEMM：当一个warpgroup的GEMM运行时，应调度另一个warpgroup的softmax。相同颜色表示同一迭代。

注意力变体 对于多查询注意力[51]和分组查询注意力[3]，我们遵循方法i
FlashAttention-2并调整张量索引，以避免在HBM中重复存储 $\mathbf{K}$ 和 $\mathbf{V}$ 。

3.2 内部warpgroup重叠的GEMM与softmax

即使在同一个warpgroup内，我们也可以将softmax中的一些指令与GEMM中的一些指令重叠执行。我们描述一种实现此目的的技术。

在注意力算法中，内层循环（主循环）内的操作存在顺序依赖，阻碍了单次迭代内的并行化。例如，（局部）softmax（第18至19行）依赖于第一次GEMM的输出 $\mathbf{S}_i^{(j)}$ ，而第二次GEMM则以其结果 $\tilde{\mathbf{P}}_i^{(j)}$ 作为操作数。实际上，算法1第17行和第21行的等待语句将softmax和GEMM的执行串行化了。然而，我们可以通过在寄存器中使用额外缓冲区，跨迭代进行流水线处理来打破这些依赖。基于这一思路，我们提出了以下两阶段⁶ GEMM-softmax流水线算法：

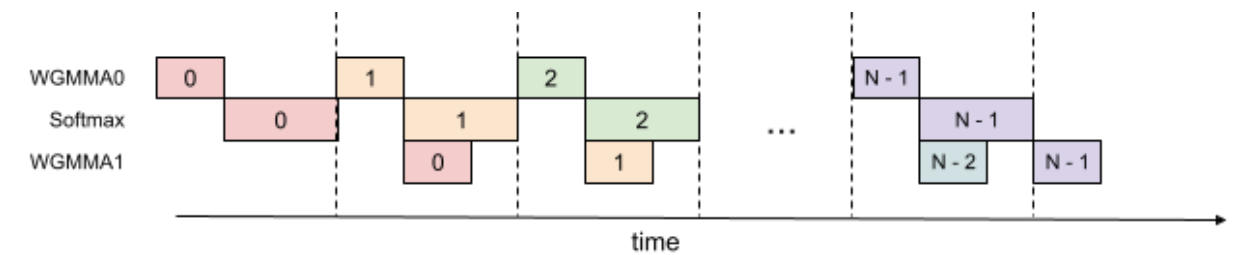


图2: 两阶段WGMMMA-softmax流水线处理

⁶Note that the number of stages of the overlapping scheme is bounded by, but need not equal, the number s of stages in the circular SMEM buffer.

Algorithm 2 FLASHATTENTION-3 consumer warpgroup forward pass

Require: Matrices $\mathbf{Q}_i \in \mathbb{R}^{B_r \times d}$ and $\mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, key block size B_c with $T_c = \lceil \frac{N}{B_c} \rceil$.

- 1: Reallocate predetermined number of registers as function of number of consumer warps.
 - 2: On-chip, initialize $\mathbf{O}_i = (0) \in \mathbb{R}^{B_r \times d}$ and $\ell_i, m_i = (0), (-\infty) \in \mathbb{R}^{B_r}$.
 - 3: Wait for $\mathbf{Q}_i$ and $\mathbf{K}_0$ to be loaded in shared memory.
 - 4: Compute $\mathbf{S}_{\text{cur}} = \mathbf{Q}_i \mathbf{K}_0^T$ using WGMMA. Commit and wait.
 - 5: Release the 0th stage of the buffer for $\mathbf{K}$.
 - 6: Compute $m_i, \tilde{\mathbf{P}}_{\text{cur}}$ and ℓ_i based on $\mathbf{S}_{\text{cur}}$, and rescale $\mathbf{O}_i$.
 - 7: **for** $1 \leq j < T_c - 1$ **do**
 - 8: Wait for $\mathbf{K}_j$ to be loaded in shared memory.
 - 9: Compute $\mathbf{S}_{\text{next}} = \mathbf{Q}_i \mathbf{K}_j^T$ using WGMMA. Commit but do not wait.
 - 10: Wait for $\mathbf{V}_{j-1}$ to be loaded in shared memory.
 - 11: Compute $\mathbf{O}_i = \mathbf{O}_i + \tilde{\mathbf{P}}_{\text{cur}} \mathbf{V}_{j-1}$ using WGMMA. Commit but do not wait.
 - 12: Wait for the WGMMA $\mathbf{Q}_i \mathbf{K}_j^T$.
 - 13: Compute $m_i, \tilde{\mathbf{P}}_{\text{next}}$ and ℓ_i based on $\mathbf{S}_{\text{next}}$.
 - 14: Wait for the WGMMA $\tilde{\mathbf{P}}_{\text{cur}} \mathbf{V}_{j-1}$ and then rescale $\mathbf{O}_i$
 - 15: Release the $(j \% s)$ th, resp. $(j - 1 \% s)$ th stage of the buffer for $\mathbf{K}$, resp. $\mathbf{V}$.
 - 16: Copy $\mathbf{S}_{\text{next}}$ to $\mathbf{S}_{\text{cur}}$.
 - 17: **end for**
 - 18: Wait for $\mathbf{V}_{T_c-1}$ to be loaded in shared memory.
 - 19: Compute $\mathbf{O}_i = \mathbf{O}_i + \tilde{\mathbf{P}}_{\text{last}} \mathbf{V}_{T_c-1}$ using WGMMA. Commit and wait.
 - 20: Epilogue: Rescale $\mathbf{O}_i$ based on m_i . Compute L_i based on m_i and ℓ_i . Write $\mathbf{O}_i$ and L_i to HBM as the i -th block of $\mathbf{O}$ and L .
-

Algorithm 2 functions as a replacement for the consumer path of Algorithm 1 to comprise the complete FLASHATTENTION-3 algorithm for FP16 precision. At a high-level, we use WGMMA as a metonym for asynchronous GEMM. Within the mainloop (lines 8 to 16), the second WGMMA operation of iteration j (line 11) is overlapped with softmax operations from iteration $j + 1$ (line 13).

While the pipelined structure illustrated above offers theoretical performance gains, there are several practical aspects to consider:

Compiler reordering The pseudocode represents an idealized execution order but the compiler (NVCC) often rearranges instructions for optimization. This can disrupt the carefully crafted WGMMA and non-WGMMA operation pipelining sequence, potentially leading to unexpected behavior or diminished performance gains. An analysis of the SASS code shows that the compiler generates overlapped code as expected (Section B.2).

Register pressure To maintain optimal performance, register spilling should be minimized. However, the 2-stage pipeline requires additional registers to store intermediate results and maintain context between stages. Specifically, an extra $\mathbf{S}_{\text{next}}$ must be kept in registers, leading to extra register usage of size $B_r \times B_c \times \text{sizeof(float)}$ per threadblock. This increased register demand may conflict with using larger block sizes (another common optimization), which is also register-hungry. In practice, trade-offs should be made based on profiling results.

3-stage pipelining Extending the 2-stage algorithm described above, we propose a 3-stage variant that would further overlap the second WGMMA with softmax. While this approach offers the potential for even higher Tensor Core utilization, it requires even more registers due to an additional stage in the pipeline, making the trade-off between tile size and pipeline depth more difficult to balance. A detailed description of the 3-stage algorithm and its evaluation results can be found in Appendix B.3.

3.3 Low-precision with FP8

Efficiency: layout transformations. Computing the forward pass of FLASHATTENTION-3 in FP8 precision poses additional challenges not encountered for FP16 in terms of layout conformance.

Algorithm 2 FLASHATTENTION-3 consumer warpgroup forward pass

Require: Matrices $\mathbf{Q}_i \in \mathbb{R}^{B_r \times d}$ and $\mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, key block size B_c with $T_c = \lceil \frac{N}{B_c} \rceil$.

- 1: Reallocate predetermined number of registers as function of number of consumer warps.
 - 2: On-chip, initialize $\mathbf{O}_i = (0) \in \mathbb{R}^{B_r \times d}$ and $\ell_i, m_i = (0), (-\infty) \in \mathbb{R}^{B_r}$.
 - 3: Wait for $\mathbf{Q}_i$ and $\mathbf{K}_0$ to be loaded in shared memory.
 - 4: Compute $\mathbf{S}_{\text{cur}} = \mathbf{Q}_i \mathbf{K}_0^T$ using WGMMA. Commit and wait.
 - 5: Release the 0th stage of the buffer for $\mathbf{K}$.
 - 6: Compute $m_i, \tilde{\mathbf{P}}_{\text{cur}}$ and ℓ_i based on $\mathbf{S}_{\text{cur}}$, and rescale $\mathbf{O}_i$.
 - 7: **for** $1 \leq j < T_c - 1$ **do**
 - 8: Wait for $\mathbf{K}_j$ to be loaded in shared memory.
 - 9: Compute $\mathbf{S}_{\text{next}} = \mathbf{Q}_i \mathbf{K}_j^T$ using WGMMA. Commit but do not wait.
 - 10: Wait for $\mathbf{V}_{j-1}$ to be loaded in shared memory.
 - 11: Compute $\mathbf{O}_i = \mathbf{O}_i + \tilde{\mathbf{P}}_{\text{cur}} \mathbf{V}_{j-1}$ using WGMMA. Commit but do not wait.
 - 12: Wait for the WGMMA $\mathbf{Q}_i \mathbf{K}_j^T$.
 - 13: Compute $m_i, \tilde{\mathbf{P}}_{\text{next}}$ and ℓ_i based on $\mathbf{S}_{\text{next}}$.
 - 14: Wait for the WGMMA $\tilde{\mathbf{P}}_{\text{cur}} \mathbf{V}_{j-1}$ and then rescale $\mathbf{O}_i$
 - 15: Release the $(j \% s)$ th, resp. $(j - 1 \% s)$ th stage of the buffer for $\mathbf{K}$, resp. $\mathbf{V}$.
 - 16: Copy $\mathbf{S}_{\text{next}}$ to $\mathbf{S}_{\text{cur}}$.
 - 17: **end for**
 - 18: Wait for $\mathbf{V}_{T_c-1}$ to be loaded in shared memory.
 - 19: Compute $\mathbf{O}_i = \mathbf{O}_i + \tilde{\mathbf{P}}_{\text{last}} \mathbf{V}_{T_c-1}$ using WGMMA. Commit and wait.
 - 20: Epilogue: Rescale $\mathbf{O}_i$ based on m_i . Compute L_i based on m_i and ℓ_i . Write $\mathbf{O}_i$ and L_i to HBM as the i -th block of $\mathbf{O}$ and L .
-

算法2作为算法1中消费者路径的替代，共同构成了完整的FP16精度FlashAttention-3算法。在高层次上，我们使用WGMMA作为异步GEMM的转喻。在主循环中（第8至16行），迭代 的第二WGMMA操作（第11行）与迭代+1的softmax操作（第13行）重叠执行。

虽然上述流水线结构在理论上能带来性能提升，但实际应用中仍需考虑以下几个因素：

编译器重排序 伪代码代表了一种理想化的执行顺序，但编译器（NVCC）通常会为了优化而重新排列指令。这可能会破坏精心设计的WGMMA和非WGMMA操作流水线序列，可能导致意外行为或性能提升减弱。对SASS代码的分析表明，编译器按预期生成了重叠代码（见B.2节）。

寄存器压力 为保持最佳性能，应尽量减少寄存器溢出。然而，两阶段流水线需要额外的寄存器来存储中间结果并维持阶段间的上下文。具体而言，必须额外保留一个 $\mathbf{S}_{\text{next}}$ 在寄存器中，导致每个线程块增加 $B_r \times B_c \times \text{sizeof(float)}$ 的寄存器使用量。这种增加的寄存器需求可能与使用更大的块尺寸（另一种常见优化）产生冲突，后者同样对寄存器需求较高。在实践中，应根据性能分析结果进行权衡。

三级流水线扩展上述的两级算法，我们提出了一个三级变体，它将进一步重叠第二次WGMMA与softmax操作。虽然这种方法有潜力实现更高的张量核心利用率，但由于流水线中增加了一个额外阶段，需要更多的寄存器，使得瓦片大小与流水线深度之间的权衡更加难以平衡。关于三级算法及其评估结果的详细描述，请参见附录B.3。

3.3 采用FP8的低精度计算

效率：布局转换。在FP8精度下计算FlashAttention-3的前向传递，在布局一致性方面带来了FP16中未遇到的额外挑战。

T0 {d0, d1}	T1 {d0, d1}	T2 {d0, d1}	T3 {d0, d1}	T0 {d4, d5}	T1 {d4, d5}	T2 {d4, d5}	T3 {d4, d5}
T0 {d2, d3}	T1 {d2, d3}	T2 {d2, d3}	T3 {d2, d3}	T0 {d6, d7}	T1 {d6, d7}	T2 {d6, d7}	T3 {d6, d7}

Figure 3: FP32 accumulator register WGMMMA layout – rows 0 and 8, threads 0-3, entries 0-7.

T0 {a0, a1}	T0 {a2, a3}	T1 {a0, a1}	T1 {a2, a3}	T2 {a0, a1}	T2 {a2, a3}	T3 {a0, a1}	T3 {a2, a3}
T0 {a4, a5}	T0 {a6, a7}	T1 {a4, a5}	T1 {a6, a7}	T2 {a4, a5}	T2 {a6, a7}	T3 {a4, a5}	T3 {a6, a7}

Figure 4: FP8 operand A register WGMMMA layout – rows 0 and 8, threads 0-3, entries 0-7.

First, we note that the input tensors $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$ are typically given as contiguous in the head dimension, while to satisfy the k-major constraint on FP8 WGMMMA for the second GEMM we need $\mathbf{V}$, or rather the tiles of $\mathbf{V}$ loaded into SMEM, to be contiguous in the sequence length dimension. Since the TMA load itself cannot change the contiguous dimension, we then need to either (1) transpose $\mathbf{V}$ in GMEM as a pre-processing step, or (2) do an in-kernel transpose of tiles of $\mathbf{V}$ after loading them into SMEM. To implement option (1), we can either (1a) fuse the transpose to the epilogue of a preceding step such as the rotary embedding, or (1b) call a standalone pre-processing transpose kernel⁷ to exchange the strides of the sequence length and head dimensions. However, (1a) is difficult to integrate into a standard library, and (1b) is too wasteful in a memory-bound situation such as inference.

Instead, for FP8 FLASHATTENTION-3 we opt for option (2). For the in-kernel transpose, we take advantage of the LDSM (`ldmatrix`) and STSM (`stmatrix`) instructions, which involve a warp of threads collectively loading SMEM to RMEM and storing RMEM to SMEM at a granularity of 128 bytes.⁸ The LDSM/STSM instructions are both register efficient, allowing us to execute them in the producer warpgroup, and capable of transposing layouts when doing memory copy. Moreover, after the first iteration we can arrange for the transpose of the next $\mathbf{V}$ tile to be executed in the shadow of the two WGMMAs that involve the preceding $\mathbf{V}$ and current $\mathbf{K}$ tile.

Second, we observe that unlike with FP16, the memory layout of the FP32 accumulator of an FP8 WGMMMA is different from that assumed for its operand A when held in registers. We depict fragments of these two layouts in Fig. 3 and Fig. 4, where the entries are held in registers per thread in the listed order. By using byte permute instructions, we can then transform the first WGMMMA’s accumulator into a format suitable for the second WGMMMA, and compatibly with the layout of the $\mathbf{V}$ tile produced by the in-kernel transpose. Specifically, with reference to Fig. 3, we change the order in sequence to

{d0 d1 d4 d5 d2 d3 d6 d7},

and this register permutation is then replicated over every 8 bytes. In terms of the logical shape of the $\mathbf{P}$ tile, this maneuver permutes its columns (e.g., columns 0189 now become the first four columns). For WGMMMA to then compute the correct output tile, we can correspondingly arrange for the in-kernel transpose to write out a matching row permutation of the $\mathbf{V}$ tile.⁹

Accuracy: block quantization and incoherent processing. With FP8 (e4m3) format, one only uses 3 bits to store the mantissa and 4 bits for the exponent. This results in higher numerical error than FP16/BF16. Moreover, large models typically have outlier values [20, 54] that are much larger in magnitude than most other values, making quantization difficult. One typically use per-tensor scaling [37] by keeping one scalar per tensor (e.g., one for $\mathbf{Q}$, for $\mathbf{K}$, and for $\mathbf{V}$). To reduce the numerical error of attention in FP8, we employ two techniques:

1. **Block quantization:** we keep one scalar per block, so that for each of $\mathbf{Q}$, $\mathbf{K}$, $\mathbf{V}$ we split the tensor into blocks of size $B_r \times d$ or $B_c \times d$ and quantize them separately. This quantization can be fused with an operation right before attention (e.g., rotary embedding) with no additional slow down (since rotary embedding is memory-bandwidth bound). As the FLASHATTENTION-3 algorithm naturally operates on blocks, we can scale each block of $\mathbf{S}$ to account for this block quantization at no computation cost.

⁷An optimized transpose kernel will achieve speed near the bandwidth of the device [46].

⁸In the PTX documentation, LDSM/STSM are described as copying 8×8 matrices with 16-bit entries [40, §9.7.13.4.15-16], but we can pack 8-bit entries two at a time to use LDSM/STSM in the context of FP8 precision. However, the transpose versions of LDSM/STSM cannot split packed 8-bit entries, which necessitates certain register movements in between LDSM and STSM to actually perform a tile-wise transpose; we omit the details.

⁹This additional freedom afforded by doing the in-kernel transpose eliminates having to use shuffle instructions to change register ownership across threads, which we previously described in [7].

T0 {d0, d1}	T1 {d0, d1}	T2 {d0, d1}	T3 {d0, d1}	T0 {d4, d5}	T1 {d4, d5}	T2 {d4, d5}	T3 {d4, d5}
T0 {d2, d3}	T1 {d2, d3}	T2 {d2, d3}	T3 {d2, d3}	T0 {d6, d7}	T1 {d6, d7}	T2 {d6, d7}	T3 {d6, d7}

图3: FP32累加器寄存器WGMMMA布局——第0行和第8行, 线程0-3, 条目0-7。

T0 {a0, a1}	T0 {a2, a3}	T1 {a0, a1}	T1 {a2, a3}	T2 {a0, a1}	T2 {a2, a3}	T3 {a0, a1}	T3 {a2, a3}
T0 {a4, a5}	T0 {a6, a7}	T1 {a4, a5}	T1 {a6, a7}	T2 {a4, a5}	T2 {a6, a7}	T3 {a4, a5}	T3 {a6, a7}

图4: FP8操作数A寄存器WGMMMA布局——第0行和第8行, 线程0-3, 条目0-7。

首先, 我们注意到输入张量 $\mathbf{Q}$ 、 $\mathbf{K}$ 和 $\mathbf{V}$ 通常在头维度上是连续的, 而为满足第二个GEMM中FP8 WGMMMA的k主序约束, 我们需要 $\mathbf{V}$ ——更准确地说, 是加载到SMEM中的 $\mathbf{V}$ 分块——在序列长度维度上连续。由于TMA加载本身无法改变连续维度, 我们需要选择以下方案: (1) 在GMEM中对 $\mathbf{V}$ 进行转置作为预处理步骤, 或(2) 在将 $\mathbf{V}$ 分块加载到SMEM后执行核内转置。为实现方案(1), 我们可以选择(1a)将转置操作融合到前序步骤(如旋转嵌入)的收尾阶段, 或(1b)调用独立的预处理转置内核⁷来交换序列长度维度与头维度的步长。然而, (1a)难以集成到标准库中, 而(1b)在推理等内存受限场景中会造成显著资源浪费。

相反, 对于FP8 FlashAttention-3, 我们选择了方案(2)。对于内核内的转置操作, 我们利用了LDSM (`ldmatrix`) 和STSM (`stmatrix`) 指令, 这些指令涉及一个线程束(warp)以128字节的粒度集体将SMEM加载到RMEM以及将RMEM存储到SMEM。⁸ LDSM/STSM指令在寄存器使用上都很高效, 使我们能够在生产者线程束组(warpgroup)中执行它们, 并且能够在内存复制时转置布局。此外, 在第一次迭代之后, 我们可以安排下一个 $\mathbf{V}$ 图块的转置操作, 在与前一个 $\mathbf{V}$ 和当前 $\mathbf{K}$ 图块相关的两个WGMMMA指令的“阴影”下执行。

其次, 我们观察到与FP16不同, FP8 WGMMMA的FP32累加器的内存布局与其操作数A在寄存器中存储时假定的布局存在差异。我们在图3和图4中展示了这两种布局的片段, 其中条目按照所列顺序存储在每个线程的寄存器中。通过使用字节置换指令, 我们可以将第一个WGMMMA的累加器转换为适合第二个WGMMMA的格式, 并与内核内转置产生的 $\mathbf{V}$ 图块布局兼容。具体而言, 参考图3, 我们将顺序更改为

{d0 d1 d4 d5 d2 d3 d6 d7},

并且这种寄存器置换随后在每个8字节上复制。就 $\mathbf{P}$ 瓦片的逻辑形状而言, 这种操作置换其列(例如, 列0189现在成为前四列)。为了让WGMMMA随后计算正确的输出瓦片, 我们可以相应地安排内核内转置, 以写出与 $\mathbf{V}$ 瓦片匹配的行置换。⁹

精度: 块量化和非相干处理。使用FP8 (e4m3) 格式时, 仅用3位存储尾数, 4位存储指数。这导致比FP16/BF16更高的数值误差。此外, 大型模型通常存在异常值[20, 54], 其数量级远大于大多数其他值, 使得量化变得困难。通常采用每张量缩放[37]的方法, 即每个张量保留一个标量(例如, 为 $\mathbf{Q}$ 、 $\mathbf{K}$ 和 $\mathbf{V}$ 各保留一个)。为了减少FP8中注意力的数值误差, 我们采用了两种技术:

1. 块量化: 我们为每个块保留一个标量, 因此对于每个 $\mathbf{Q}$ 、 $\mathbf{K}$ 、 $\mathbf{V}$, 我们将张量分割成大小为 $r \times$ 或 $c \times$ 的块, 并分别对它们进行量化。这种量化可以与注意力操作之前的步骤(例如, 旋转嵌入)融合, 不会造成额外的减速(因为旋转嵌入受内存带宽限制)。由于FlashAttention-3算法天然在块上操作, 我们可以缩放每个 $\mathbf{S}$ 的块来适应这种块量化, 而无需计算成本。

⁷An optimized transpose kernel will achieve speed near the bandwidth of the device [46].

⁸In the PTX documentation, LDSM/STSM are described as copying 8×8 matrices with 16-bit entries [40, §9.7.13.4.15-16], but we can pack 8-bit entries two at a time to use LDSM/STSM in the context of FP8 precision. However, the transpose versions of LDSM/STSM cannot split packed 8-bit entries, which necessitates certain register movements in between LDSM and STSM to actually perform a tile-wise transpose; we omit the details.

⁹This additional freedom afforded by doing the in-kernel transpose eliminates having to use shuffle instructions to change register ownership across threads, which we previously described in [7].

2. **Incoherent processing:** to even out outliers, we multiply $\mathbf{Q}$ and $\mathbf{K}$ with a random orthogonal matrix $\mathbf{M}$ before quantizing to FP8. Since $\mathbf{M}$ is orthogonal, $\mathbf{MM}^\top = \mathbf{I}$ and so $(\mathbf{QM})(\mathbf{KM})^\top = \mathbf{QK}^\top$, i.e., multiplying both $\mathbf{Q}$ and $\mathbf{K}$ with $\mathbf{M}$ does not change the attention output. This serves to “spread out” the outliers since each entry of $\mathbf{QM}$ or $\mathbf{KM}$ is a random sum of entries of $\mathbf{Q}$ or $\mathbf{K}$, thus reducing quantization error. In practice, we follow Chee et al. [9] and Tseng et al. [58] and choose $\mathbf{M}$ to be the product of random diagonal matrices of ± 1 and a Hadamard matrix, which can be multiplied in $O(d \log d)$ instead of $O(d^2)$, and can also be fused with the rotary embedding at no extra computation cost.

We validate that these two techniques reduces numerical error by up to 2.6 $\times$ in §4.3.

4 Empirical Validation

We use the primitives from CUTLASS [57] such as WGMMMA and TMA abstractions to implement FLASHATTENTION-3 and evaluate its efficiency and accuracy.

- **Benchmarking attention.** We measure the runtime of FLASHATTENTION-3 across different sequence lengths and compare it to a standard implementation in PyTorch, FLASHATTENTION-2, FLASHATTENTION-2 in Triton (which uses H100-specific instructions), as well as a vendor’s implementation of FLASHATTENTION-2 optimized for H100 GPUs from cuDNN. We confirm that FLASHATTENTION-3 is up to 2.0 $\times$ faster than FLASHATTENTION-2 and 1.5 $\times$ faster than FLASHATTENTION-2 in Triton. FLASHATTENTION-3 reaches up to 740 TFLOPs/s, 75% of the theoretical maximum TFLOPs/s on H100 GPUs.
- **Ablation study.** We confirm that our algorithmic improvements with warp-specialization and GEMM-softmax pipelining contribute to the speedup of FLASHATTENTION-3.
- **Accuracy of FP8 attention.** We validate that block quantization and incoherent processing reduces the numerical error of FP8 FLASHATTENTION-3 by 2.6 $\times$.

4.1 Benchmarking Attention

We measure the runtime of different attention methods on an H100 80GB SXM5 GPU for different settings (without / with causal mask, head dimension 64 or 128) for FP16 inputs. We report the results in Fig. 5 and Fig. 6, showing that FLASHATTENTION-3 is around 1.5-2.0 $\times$ faster than FLASHATTENTION-2 in the forward pass and 1.5-1.75 $\times$ faster in the backward pass. Compared to a standard attention implementation, FLASHATTENTION-3 can be up to 3-16 $\times$ faster. For medium and long sequences (1k and above), FLASHATTENTION-3 even surpasses the speed of a vendor’s library (cuDNN – closed source) that has been optimized for H100 GPUs.

Benchmark settings: We vary the sequence length as 512, 1k, ..., 16k, and set batch size so that the total number of tokens is 16k. We set the hidden dimension to 2048, and head dimension to be either 64, 128, or 256 (i.e., 32 heads, 16 heads, or 8 heads). To calculate the FLOPs of the forward pass, we use:

$$4 \cdot \text{seqlen}^2 \cdot \text{head dimension} \cdot \text{number of heads}.$$

With causal masking, we divide this number by 2 to account for the fact that approximately only half of the entries are calculated. To get the FLOPs of the backward pass, we multiply the forward pass FLOPs by 2.5 (since there are 2 matmuls in the forward pass and 5 matmuls in the backward pass, due to recomputation).

We also measure the runtime for FP8 for the forward pass under similar settings. We report the results for headdim 256 in Fig. 7 and give the full results in Appendix C.2.

4.2 Ablation Study: 2-Stage Pipelining Experiments

We ablate both the 2-stage WGMMMA-softmax pipelining and warp-specialization for non-causal FP16 FLASHATTENTION-3 with fixed parameters {batch, seqlen, nheads, hdim} = {4, 8448, 16, 128}. The result in Table 2 confirms that our algorithmic improvements (asynchrony with warp-specialization and overlapping between GEMM and softmax) lead to significant speedup, from 570 to 661 TFLOPs.

2. 非相干处理：为消除异常值的影响，我们在量化至FP8前将 $\mathbf{Q}$ 和 $\mathbf{K}$ 与随机正交矩阵 $\mathbf{M}$ 相乘。由于 $\mathbf{M}$ 是正交矩阵，满足 $\mathbf{MM}^\top = \mathbf{I}$ ，因此 $(\mathbf{QM})(\mathbf{KM})^\top = \mathbf{QK}^\top$ ，即用 $\mathbf{M}$ 同时乘以 $\mathbf{Q}$ 和 $\mathbf{K}$ 不会改变注意力输出结果。这种方法能“分散”异常值，因为 $\mathbf{QM}$ 或 $\mathbf{KM}$ 的每个元素都是 $\mathbf{Q}$ 或 $\mathbf{K}$ 元素的随机加权和，从而降低量化误差。实际应用中，我们遵循Chee等人[9]和Tseng等人[58]的方法，选择 $\mathbf{M}$ 作为 ± 1 的随机对角矩阵与哈达玛矩阵的乘积，该运算复杂度为 $(\log d)$ 而非 (d^2) ，且可与旋转嵌入融合而无需额外计算开销。

我们验证了这两种技术在§4.3中将数值误差降低了高达2.6 $\times$ 。

4 实证验证

我们利用CUTLASS [57]中的WGMMMA和TMA抽象等原语来实现FlashAttention。3，并评估其效率和准确性。

- 基准测试注意力机制。我们测量了FlashAttention-3在不同序列长度下的运行时间，并将其与PyTorch中的标准实现、FlashAttention-2、Triton中的FlashAttention-2（使用H100专用指令）以及cuDNN为H100 GPU优化的FlashAttention-2供应商实现进行了比较。我们确认FlashAttention-3比FlashAttention-2快达2.0 $\times$ 倍，比Triton中的FlashAttention-2快1.5 $\times$ 倍。FlashAttention-3最高可达740 TFLOPs/s，达到H100 GPU理论最大TFLOPs/s的75%。
- 消融研究。我们确认，采用warp-specialization和GEMM-softmax流水线的算法改进对FlashAttention-3的加速起到了贡献。
- FP8注意力机制的准确性。我们验证了块量化和非相干处理将FP8 FlashAttention-3的数值误差降低了2.6 $\times$ 。

4.1 注意力机制基准测试

我们在H100 80GB SXM5 GPU上测量了不同注意力方法在不同设置下（有无因果掩码、头维度64或128）对FP16输入的处理时间。结果如图5和图6所示，表明FlashAttention-3在前向传播中比FlashAttention-2快约1.5-2.0 $\times$ 倍，在后向传播中快1.5-1.75 $\times$ 倍。与标准注意力实现相比，FlashAttention-3最高可提速3-16 $\times$ 倍。对于中长序列（1k及以上），FlashAttention-3甚至超越了针对H100 GPU优化的厂商闭源库（cuDNN）的运行速度。

基准设置：我们将序列长度变化为512、1k、...、16k，并设置批次大小以使总标记数为16k。我们将隐藏维度设置为2048，头部维度设置为64、128或256（即32个头、16个头或8个头）。为了计算前向传播的FLOPs，我们使用：

$$4 \cdot \text{序列长度}^2 \cdot \text{头维度} \cdot \text{头数}。$$

在因果掩码的情况下，我们将这个数字除以2，以考虑到大约只有一半的条目被计算的事实。为了得到反向传播的FLOPs，我们将前向传播的FLOPs乘以2.5（因为前向传播中有2次矩阵乘法，而反向传播中由于重计算有5次矩阵乘法）。

我们还在类似设置下测量了FP8在前向传播中的运行时间。我们报告了结果图7中的headdim为256，完整结果见附录C.2。

4.2 消融研究：两阶段流水线实验

我们针对非因果FP16 FlashAttention-3，在固定参数{批次大小、序列长度、头数、隐藏维度} = {4, 8448, 16, 128}下，对两阶段WGMMMA-softmax流水线和warp专用化进行了消融实验。表2中的结果证实，我们的算法改进（通过warp专用化实现异步操作，以及GEMM与softmax之间的重叠计算）带来了显著的加速效果，从570 TFLOPs提升至661 TFLOPs。

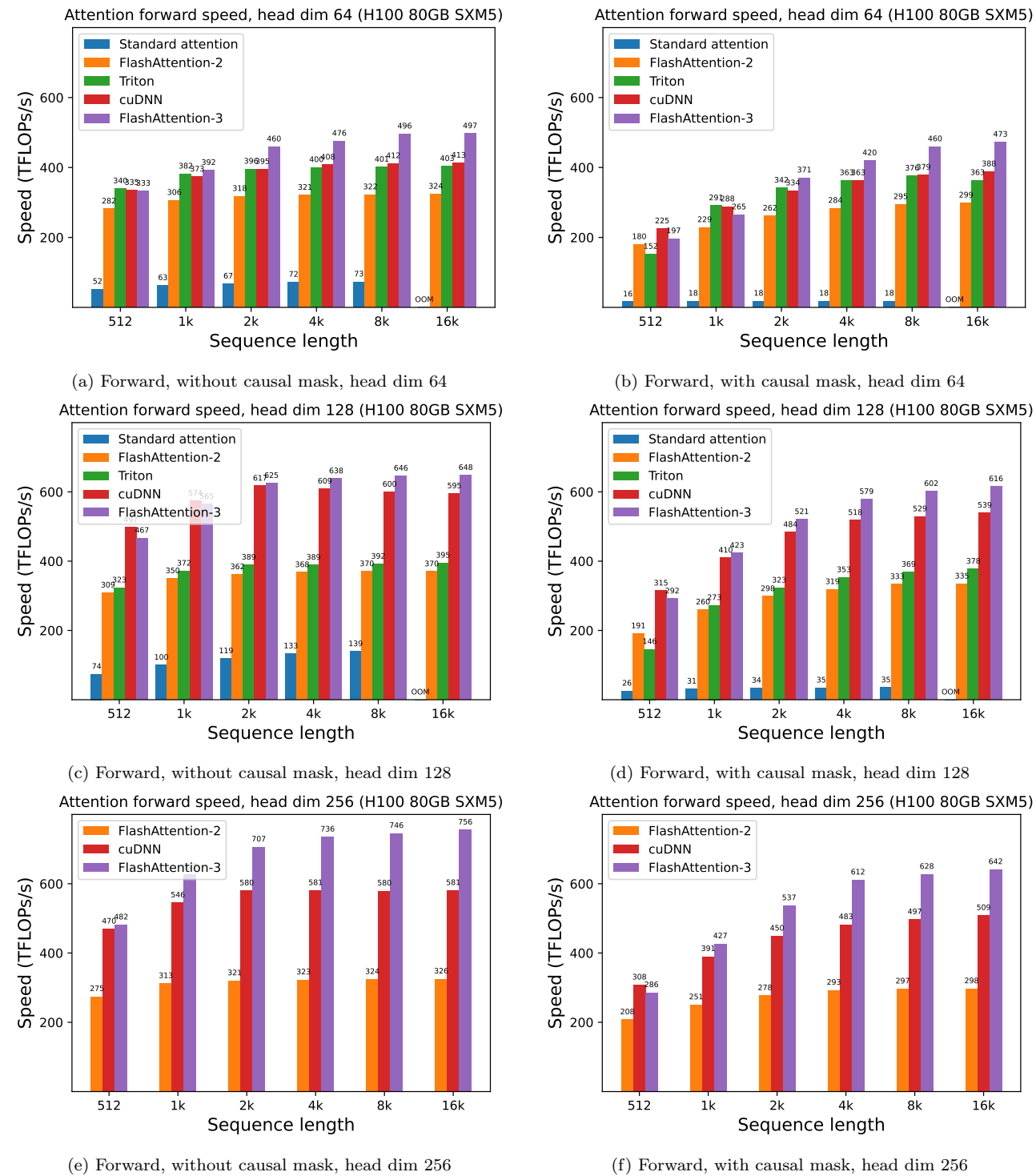


Figure 5: Attention forward speed (FP16/BF16) on H100 GPU

4.3 Numerical Error Validation

As there has been interest in the numerical error [21] of FLASHATTENTION, we compare FLASHATTENTION-2, FLASHATTENTION-3, and a standard implementation of attention against a reference implementation in FP64. To simulate outlier features and activations in LLMs [20, 54], we generate the entries of $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ with the following

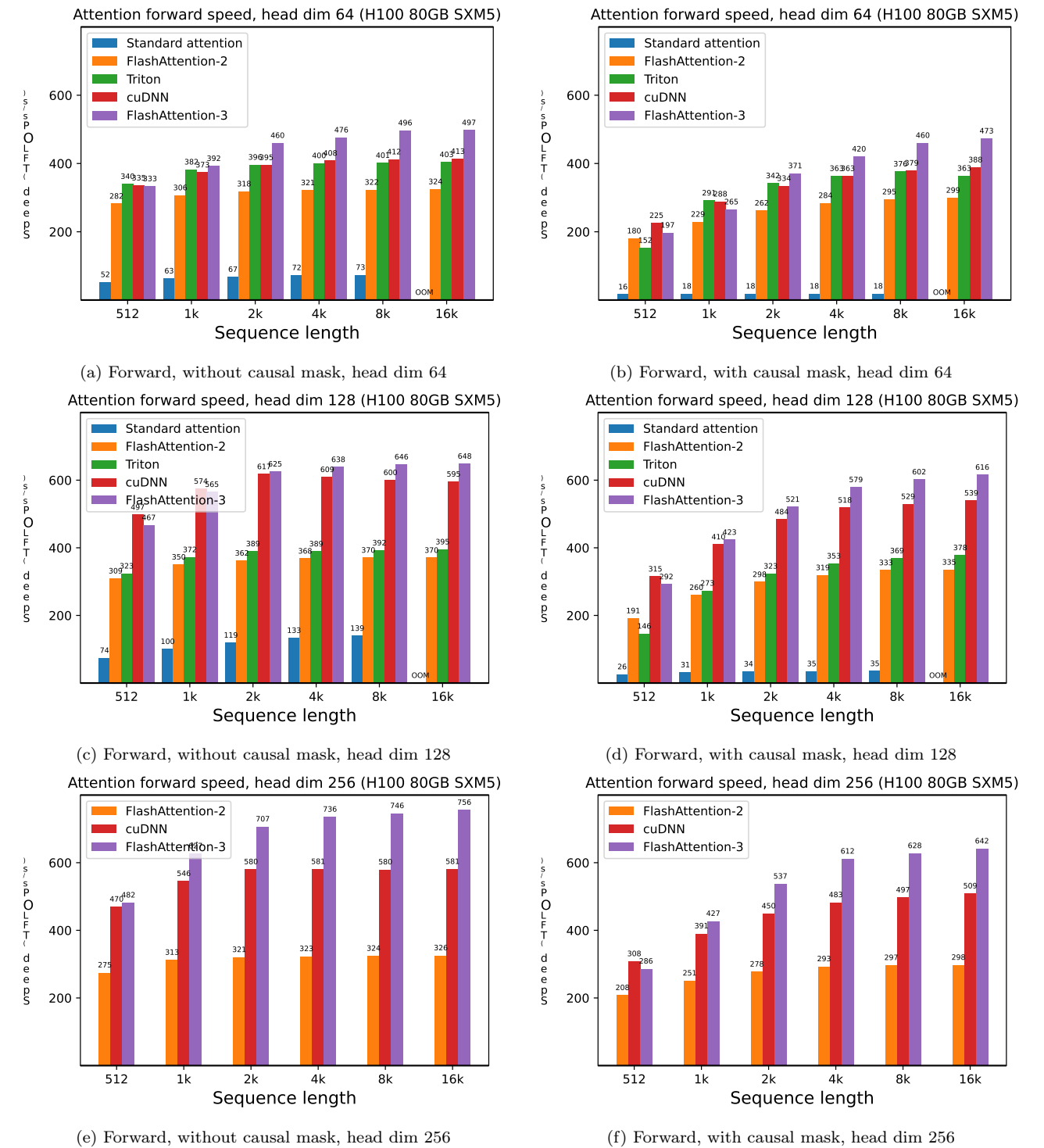


图5: H100 GPU上的注意力前向速度 (FP16/BF16)

4.3 数值误差验证

由于人们对FlashAttention的数值误差[21]感兴趣，我们将FlashAttention-2、FlashAttention-3以及标准注意力实现与FP64参考实现进行比较。为了模拟LLM中的异常特征和激活[20, 54]，我们使用以下方式生成 $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ 的条目

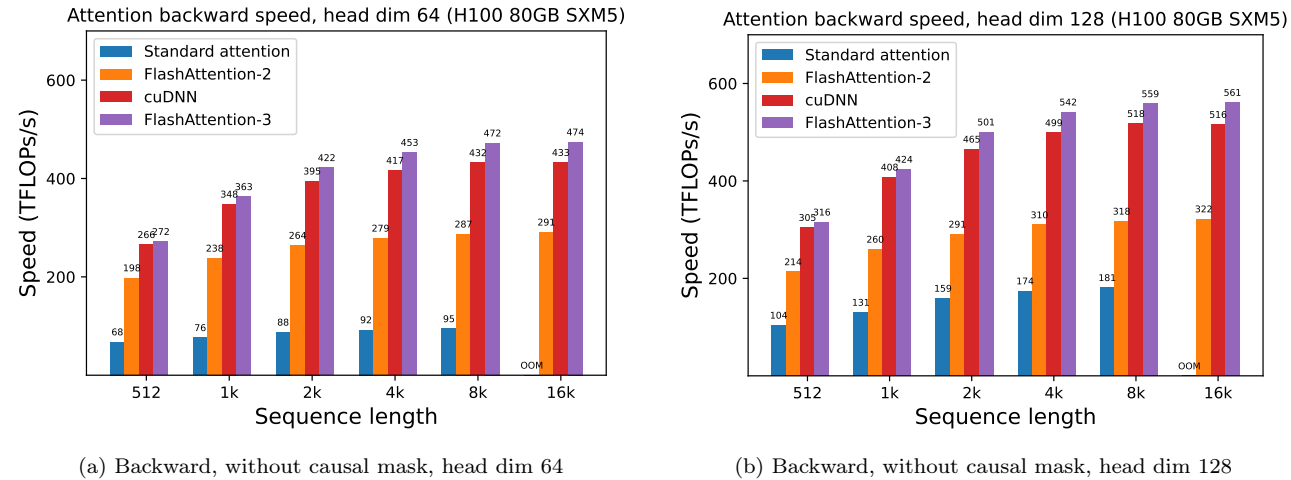


Figure 6: Attention backward speed (FP16/BF16) on H100 GPU

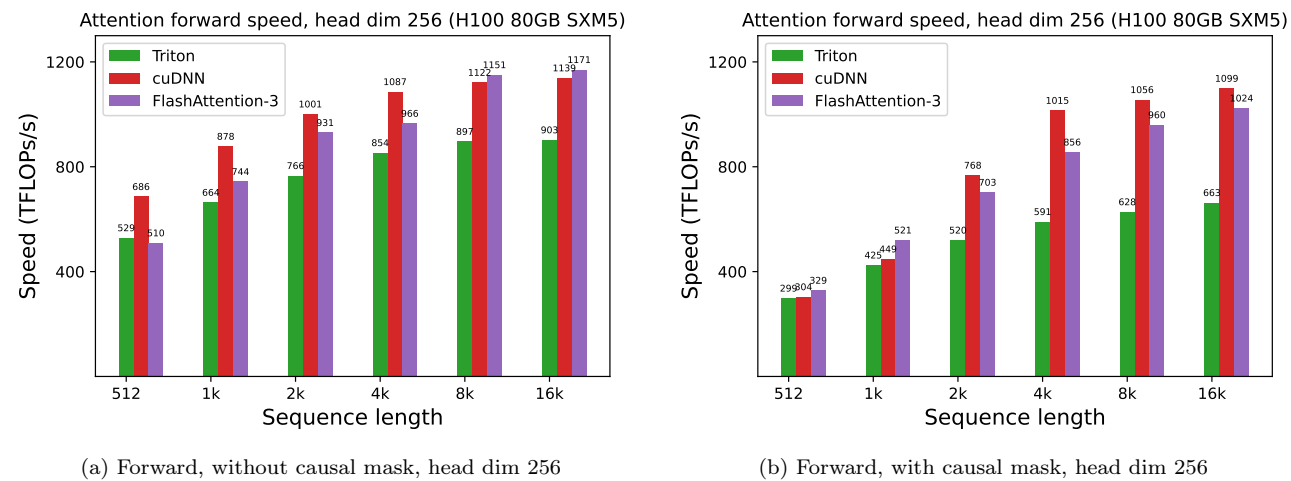


Figure 7: Attention forward speed (FP8) on H100 GPU

Table 2: Pipelining ablation measurements

Configuration	Time	TFLOPs/s
FLASHATTENTION-3	3.538 ms	661
No GEMM-Softmax Pipelining, Warp-Specialization	4.021 ms	582
GEMM-Softmax Pipelining, No Warp-Specialization	4.105 ms	570

distribution:

$$\mathcal{N}(0, 1) + \mathcal{N}(0, 100) \cdot \text{Bernoulli}(0.001).$$

That is, each entry is normally distributed with zero mean and standard deviation 1, but for 0.1% of entries we add an independent term that’s normally distributed with standard deviation 10. We then measure the root mean squared error (RMSE) in Table 3. In FP16, both FLASHATTENTION-2 and FLASHATTENTION-3 achieves 1.7× lower RMSE compared to the standard implementation since intermediate results (softmax) are kept in FP32. The baseline attention in FP8 uses per-tensor scaling, with matmul accumulator in FP32 and intermediate softmax results kept in FP16. Thanks to block quantization and incoherent processing, FLASHATTENTION-3 in FP8 is 2.6× more accurate than this baseline.

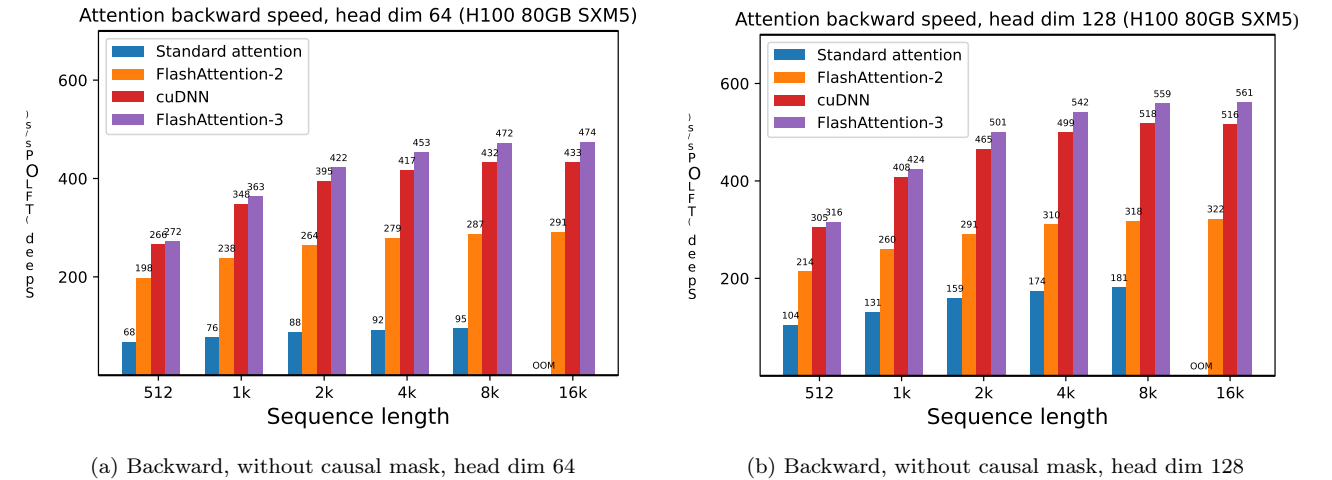


图6: H100 GPU上的注意力反向传播速度 (FP16/BF16)

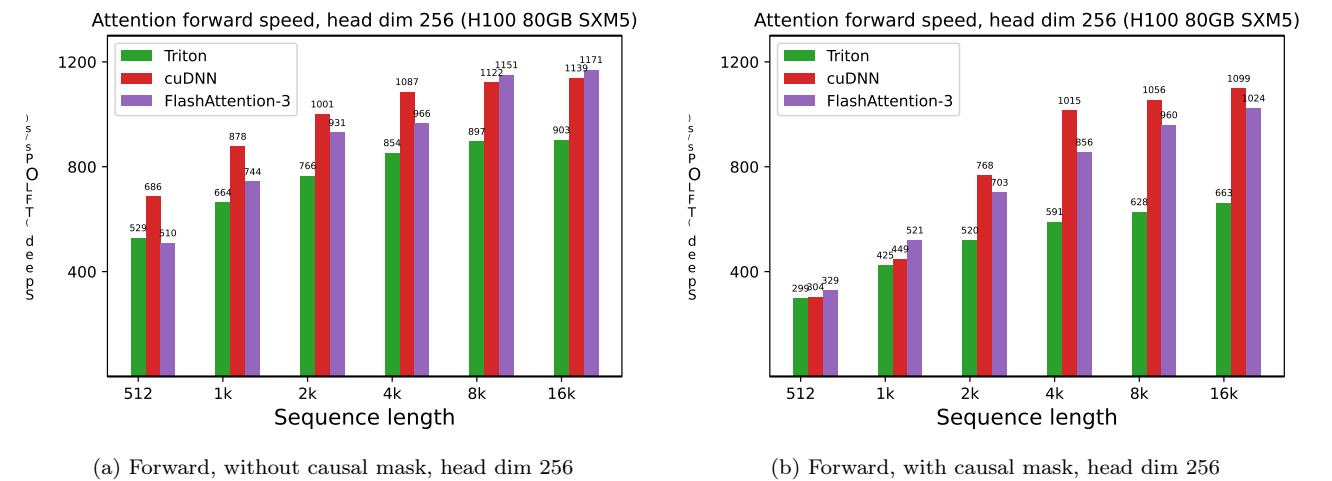


图7: H100 GPU上的注意力前向速度 (FP8)

表2: 流水线消融测量

Configuration	Time	TFLOPs/s
FLASHATTENTION-3	3.538 ms	661
No GEMM-Softmax Pipelining, Warp-Specialization	4.021 ms	582
GEMM-Softmax Pipelining, No Warp-Specialization	4.105 ms	570

分布:

$$\mathcal{N}(0, 1) + \mathcal{N}(0, 100) \cdot \text{Bernoulli}(0.001).$$

也就是说，每个条目通常服从均值为零、标准差为1的正态分布，但对于0.1%的条目，我们会添加一个独立项，该项服从标准差为10的正态分布。随后，我们在表3中测量了均方根误差（RMSE）。在FP16精度下，由于中间结果（softmax）保持在FP32，FlashAttention-2和FlashAttention-3的RMSE相比标准实现降低了1.7×。FP8中的基线注意力采用逐张量缩放，矩阵乘法累加器使用FP32，中间softmax结果保持在FP16。得益于块量化和非相干处理，FP8下的FlashAttention-3比该基线准确度提高了2.6×。

Table 3: Numerical error comparisons in FP16 and FP8 (e4m3).

Method	Baseline FP16	FLASHATTENTION-2 FP16	FLASHATTENTION-3 FP16
	RMSE	3.2e-4	1.9e-4
Method	Baseline FP8	FLASHATTENTION-3 FP8	No block quant
RMSE	2.4e-2	9.1e-3	9.3e-3
			No incoherent processing
			2.4e-2

5 Dicussion, Limitations, Conclusion

With FLASHATTENTION-3, we have demonstrated that new programming techniques and hardware features such as asynchrony and low-precision can have a dramatic impact on the efficiency and accuracy of attention. We are able to speed up attention by 1.5-2.0× times compared to FLASHATTENTION-2, and reduce FP8 numerical error by 2.6× compared to standard per-tensor quantization. Some limitations of our work that we hope to address in the future include: optimizing for LLM inference, integrating a persistent kernel design into the FP8 kernel,¹⁰ and understanding the effects of low-precision attention in large-scale training. Though we have focused on Hopper GPUs in this work, we expect that the techniques developed here will apply to other hardware accelerators. We hope that a faster and more accurate primitive such as attention will unlock new applications in long-context tasks.

Acknowledgments

We are grateful to the NVIDIA CUTLASS team (especially Haicheng Wu, Aniket Shivam, and Cris Cecka) for helping us understand Hopper’s programming model and for their library, which provides clean and powerful building blocks for the implementation of FLASHATTENTION-3. We thank the cuDNN team for the idea of in-kernel transpose for FP8. The idea of overlapping GEMMs and softmax was inspired by insightful conversations with Christopher Ré, Benjamin Spector, Aniket Shivam, and Markus Hoehnerbach. The pingpong scheduling is adapted from the warp-specialized pingpong GEMM implementation in CUTLASS. We appreciate Driss Guessous for integrating FLASHATTENTION to PyTorch. FLASHATTENTION-3 has benefited from helpful discussions with Horace He on different attention variants, with Hao Liu and Phil Wang on distributed attention, and with Daniel Haziza and Chris De Sa on quantization. We thank Meta, Together AI, and Princeton Language and Intelligence (PLI) for compute support.

References

- [1] Ahmad Abdelfattah, Azzam Haidar, Stanimire Tomov, and Jack Dongarra. Performance, design, and autotuning of batched gemm for gpus. pages 21–38, 06 2016. ISBN 978-3-319-41320-4. doi: 10.1007/978-3-319-41321-1_2.
- [2] AI21. Introducing jamba: Ai21’s groundbreaking ssm-transformer model. *AI21 blog*, 2024.
- [3] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. *arXiv preprint arXiv:2305.13245*, 2023.
- [4] Michael Bauer, Henry Cook, and Brucek Khailany. CudaDMA: Optimizing GPU Memory Bandwidth via Warp Specialization. In *Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis*, SC ’11, New York, NY, USA, 2011. Association for Computing Machinery. ISBN 9781450307710. doi: 10.1145/2063384.2063400. URL <https://doi.org/10.1145/2063384.2063400>.
- [5] Maximilian Beck, Korbinian Pöppel, Markus Spanring, Andreas Auer, Oleksandra Prudnikova, Michael Kopp, Günter Klambauer, Johannes Brandstetter, and Sepp Hochreiter. xlstm: Extended long short-term memory. *arXiv preprint arXiv:2405.04517*, 2024.

¹⁰For our benchmarks, FP16 FLASHATTENTION-3 has a persistent kernel and load balancing strategy, while FP8 FLASHATTENTION-3 does not. This partly explains why FP8 FLASHATTENTION-3 does not perform as well for small sequence length and causal masking compared to the FP8 cuDNN kernels.

表3：FP16和FP8（e4m3）中的数值误差比较。

Method	Baseline FP16	FLASHATTENTION-2 FP16	FLASHATTENTION-3 FP16
	RMSE	3.2e-4	1.9e-4
Method	Baseline FP8	FLASHATTENTION-3 FP8	No block quant
RMSE	2.4e-2	9.1e-3	9.3e-3
			No incoherent processing
			2.4e-2

5 讨论、局限性与结论

通过FlashAttention-3，我们证明了异步和低精度等新编程技术和硬件特性能够显著影响注意力的效率和准确性。相比FlashAttention-2，我们实现了注意力计算速度提升1.5-2.0×倍，并将FP8数值误差较标准逐张量量化降低2.6×倍。当前工作的若干局限有待未来改进，包括：针对大语言模型推理的优化、将持久化内核设计整合至FP8内核¹⁰，以及探究低精度注意力在大规模训练中的影响。虽然本研究主要基于Hopper GPU，但我们预计相关技术可拓展至其他硬件加速器。我们相信，这种更快速、更精确的注意力原语将推动长上下文任务的新应用发展。

致谢

我们感谢NVIDIA CUTLASS团队（特别是Haicheng Wu、Aniket Shivam和Cris Cecka）帮助我们理解Hopper的编程模型，并感谢他们的库为FlashAttention-3的实现提供了简洁而强大的构建模块。我们感谢cuDNN团队提出的FP8内核内转置的想法。重叠GEMM和softmax的灵感来源于与Christopher Ré、Benjamin Spector、Aniket Shivam和Markus Hoehnerbach的深入交流。乒乓调度方案改编自CUTLASS中的warp专用乒乓GEMM实现。我们感谢Driss Guessous将FlashAttention集成到PyTorch中。FlashAttention-3受益于与Horace He关于不同注意力变体的有益讨论，与Hao Liu和Phil Wang关于分布式注意力的讨论，以及与Daniel Haziza和Chris De Sa关于量化的讨论。我们感谢Meta、Together AI和普林斯顿语言与智能研究所（PLI）提供的计算支持。

参考文献

- [1] 艾哈迈德·阿卜杜勒法塔赫、阿扎姆·海达尔、斯塔尼米尔·托莫夫与杰克·唐加拉。面向高性能计算中张量核心的性能、设计关于GPU的批量通用矩阵乘法。第21-38页，2016年6月。ISBN 978-3-319-41320-4. doi: 10.1007/978-3-319-41321-1_2.
- [2] AI21。介绍Jamba：AI21突破性的SSM-Transformer模型。AI21博客，2024年。
- [3] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, Sumit Sanghai. GQA：从多头检查点训练广义多查询Transformer模型。arXiv预印本 arXiv:2305.13245, 2023.
- [4] Michael Bauer、Henry Cook 与 Brucek Khailany。CudaDMA：通过线程束专业化优化 GPU 内存带宽。载于《2011 年国际高性能计算、网络、存储与分析会议论文集》，SC '11，美国纽约州纽约市，2011 年。计算机协会。ISBN 9781450307710. doi: 10.1145/2063384.2063400. URL <https://doi.org/10.1145/2063384.2063400>。
- [5] Maximilian Beck, Korbinian Pöppel, Markus Spanring, Andreas Auer, Oleksandra Prudnikova, Michael Kopp, Günter Klambauer, Johannes Brandstetter, 以及 Sepp Hochreiter. xlstm：扩展长短期记忆。arXiv 预印本 arXiv:2405.04517, 2024年。

¹⁰For our benchmarks, FP16 FLASHATTENTION-3 has a persistent kernel and load balancing strategy, while FP8 FLASHATTENTION-3 does not. This partly explains why FP8 FLASHATTENTION-3 does not perform as well for small sequence length and causal masking compared to the FP8 cuDNN kernels.

[6] Iz Beltagy, Matthew E Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.

[7] Ganesh Bikshandi and Jay Shah. Delivering 1 PFLOP/s of Performance with FP8 FlashAttention-2, 2024. URL <https://research.colfax-intl.com/adding-fp8-to-flashattention/>.

[8] William Brandon, Aniruddha Nrusimha, Kevin Qian, Zachary Ankner, Tian Jin, Zhiye Song, and Jonathan Ragan-Kelley. Striped attention: Faster ring attention for causal transformers. *arXiv preprint arXiv:2311.09431*, 2023.

[9] Jerry Chee, Yaohui Cai, Volodymyr Kuleshov, and Christopher M De Sa. Quip: 2-bit quantization of large language models with guarantees. *Advances in Neural Information Processing Systems*, 36, 2024.

[10] Beidi Chen, Tri Dao, Eric Winsor, Zhao Song, Atri Rudra, and Christopher Ré. Scatterbrain: Unifying sparse and low-rank attention. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[11] Richard J Chen, Chengkuan Chen, Yicong Li, Tiffany Y Chen, Andrew D Trister, Rahul G Krishnan, and Faisal Mahmood. Scaling vision transformers to gigapixel images via hierarchical self-supervised learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16144–16155, 2022.

[12] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*, 2019.

[13] Krzysztof Choromanski, Valerii Likhoshesterov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Davis, Afroz Mohiuddin, Lukasz Kaiser, et al. Rethinking attention with performers. In *The International Conference on Learning Representations (ICLR)*, 2021.

[14] Krzysztof Marcin Choromanski, Valerii Likhoshesterov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Quincy Davis, Afroz Mohiuddin, Lukasz Kaiser, et al. Rethinking attention with performers. In *International Conference on Learning Representations (ICLR)*, 2020.

[15] Tri Dao. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning, 2023. URL <https://arxiv.org/abs/2307.08691>.

[16] Tri Dao and Albert Gu. Transformers are SSMs: Generalized models and efficient algorithms with structured state space duality. In *International Conference on Machine Learning (ICML)*, 2024.

[17] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, 2022.

[18] Tri Dao, Daniel Y Fu, Khaled K Saab, Armin W Thomas, Atri Rudra, and Christopher Ré. Hungry hungry hippos: Towards language modeling with state space models. In *The International Conference on Learning Representations (ICLR)*, 2023.

[19] DeepSeek-AI. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024.

[20] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. Llm.int8(): 8-bit matrix multiplication for transformers at scale. *CoRR abs/2208.07339*, 2022.

[21] Alicia Golden, Samuel Hsia, Fei Sun, Bilge Acun, Basil Hosmer, Yejin Lee, Zachary DeVito, Jeff Johnson, Gu-Yeon Wei, David Brooks, et al. Is flash attention stable? *arXiv preprint arXiv:2405.02803*, 2024.

[22] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. 2023.

[23] Anmol Gulati, James Qin, Chung-Cheng Chiu, Niki Parmar, Yu Zhang, Jiahui Yu, Wei Han, Shibo Wang, Zhengdong Zhang, Yonghui Wu, et al. Conformer: Convolution-augmented transformer for speech recognition. *arXiv preprint arXiv:2005.08100*, 2020.

[6] Iz Beltagy、Matthew E Peters 和 Arman Cohan。Longformer：长文档 Transformer。arXiv 预印本 arXiv:2004.05150，2020年。[7] Ganesh Bikshandi 和 Jay Shah。使用 FP8 FlashAttention-2 实现 1 PFLOP/s 性能，2024年。URL <https://research.colfax-intl.com/adding-fp8-to-flashattention/>。[8] William Brandon、Aniruddha Nrusimha、Kevin Qian、Zachary Ankner、Tian Jin、Zhiye Song 和 Jonathan Ragan-Kelley。条纹注意力：因果 Transformer 的更快速环注意力。arXiv 预印本 arXiv:2311.09431，2023年。[9] Jerry Chee、Yaohui Cai、Volodymyr Kuleshov 和 Christopher M De Sa。Quip：具有保证的大语言模型 2 位量化。神经信息处理系统进展，第 36 卷，2024年。

[10] 陈北地、陶特里、埃里克·温瑟、宋钊、阿特里·鲁德拉、克里斯托弗·雷。《Scatterbrain：统一稀疏与低秩注意力》。载于《神经信息处理系统进展》（NeurIPS），2021年。

[11] Richard J Chen, Chengkuan Chen, Yicong Li, Tiffany Y Chen, Andrew D Trister, Rahul G Krishnan, 和 Faisal Mahmood。通过分层自监督学习将视觉Transformer扩展到十亿像素图像。载于《IEEE/CVF计算机视觉与模式识别会议论文集》，第16144–16155页，2022年。[12] Rewon Child, Scott Gray, Alec Radford, 和 Ilya Sutskever。使用稀疏Transformer生成序列。arXiv预印本 arXiv:1904.10509，2019年。

[13] Krzysztof Choromanski, Valerii Likhoshesterov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Davis, Afroz Mohiuddin, Lukasz Kaiser 等。重新思考注意力机制：Performers。发表于《国际学习表征会议》（ICLR），2021年。

[14] Krzysztof Marcin Choromanski, Valerii Likhoshesterov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Quincy Davis, Afroz Mohiuddin, Lukasz Kaiser 等。重新思考注意力机制：Performers。收录于国际学习表征会议（ICLR），2020年。

[15] Tri Dao。FlashAttention-2：具有更优并行性与工作划分的更快注意力机制，2023年。URL <https://arxiv.org/abs/2307.08691>。

[16] Tri Dao 和 Albert Gu。Transformer 是 SSM：具有结构性的广义模型与高效算法状态空间对偶性。发表于国际机器学习大会（ICML），2024年。

[17] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré。FlashAttention：快速且内存具有IO感知的高效精确注意力机制。收录于《神经信息处理系统进展》，2022年。

[18] Tri Dao、Daniel Y Fu、Khaled K Saab、Armin W Thomas、Atri Rudra 与 Christopher Ré。《饥饿的河马：迈向基于状态空间模型的语言建模》。收录于《国际学习表征会议》（ICLR），2023年。

[19] DeepSeek-AI。Deepseek-v2：一个强大、经济且高效的专家混合语言模型。arXiv 预印本 arXiv:2405.04434，2024。

[20] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer。LLM.int8(): 用于大规模Transformer的8位矩阵乘法。CoRR abs/2208.07339, 2022.[21] Alicia Golden, Samuel Hsia, Fei Sun, Bilge Acun, Basil Hosmer, Yejin Lee, Zachary DeVito, Jeff Johnson, Gu-Yeon Wei, David Brooks, et al. Flash Attention稳定吗? arXiv preprint arXiv:2405.02803, 2024.[22] Albert Gu and Tri Dao。Mamba: 基于选择性状态空间的线性时间序列建模。2023.[23] Anmol Gulati, James Qin, Chung-Cheng Chiu, Niki Parmar, Yu Zhang, Jiahui Yu, Wei Han, Shibo Wang, Zhengdong Zhang, Yonghui Wu, et al. Conformer: 用于语音识别的卷积增强Transformer。arXiv preprint arXiv:2005.08100, 2020。

[24] Mandy Guo, Joshua Ainslie, David Uthus, Santiago Ontanon, Jianmo Ni, Yun-Hsuan Sung, and Yinfei Yang. Longt5: Efficient text-to-text transformer for long sequences. *arXiv preprint arXiv:2112.07916*, 2021.

[25] Jonathan Ho, Tim Salimans, Alexey Gritsenko, William Chan, Mohammad Norouzi, and David J Fleet. Video diffusion models. *Advances in Neural Information Processing Systems*, 35:8633–8646, 2022.

[26] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. Kvquant: Towards 10 million context length llm inference with kv cache quantization. *arXiv preprint arXiv:2401.18079*, 2024.

[27] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *International Conference on Machine Learning*, pages 5156–5165. PMLR, 2020.

[28] Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. In *The International Conference on Machine Learning (ICML)*, 2020.

[29] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626, 2023.

[30] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. Starcoder: may the source be with you! *arXiv preprint arXiv:2305.06161*, 2023.

[31] Hao Liu, Matei Zaharia, and Pieter Abbeel. Ring attention with blockwise transformers for near-infinite context. *arXiv preprint arXiv:2310.01889*, 2023.

[32] Hao Liu, Wilson Yan, Matei Zaharia, and Pieter Abbeel. World model on million-length video and language with ringattention. *arXiv preprint arXiv:2402.08268*, 2024.

[33] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. Kivi: A tuning-free asymmetric 2bit quantization for kv cache. *arXiv preprint arXiv:2402.02750*, 2024.

[34] Weile Luo, Ruibo Fan, Zeyu Li, Dayou Du, Qiang Wang, and Xiaowen Chu. Benchmarking and Dissecting the Nvidia Hopper GPU Architecture, 2024. URL <https://arxiv.org/abs/2402.13499>.

[35] Xuezhe Ma, Chunting Zhou, Xiang Kong, Junxian He, Liangke Gui, Graham Neubig, Jonathan May, and Luke Zettlemoyer. Mega: Moving average equipped gated attention. In *The International Conference on Learning Representations (ICLR)*, 2023.

[36] Xuezhe Ma, Xiaomeng Yang, Wenhan Xiong, Beidi Chen, Lili Yu, Hao Zhang, Jonathan May, Luke Zettlemoyer, Omer Levy, and Chunting Zhou. Megalodon: Efficient llm pretraining and inference with unlimited context length. *arXiv preprint arXiv:2404.08801*, 2024.

[37] Paulius Micikevicius, Dusan Stolic, Neil Burgess, Marius Cornea, Pradeep Dubey, Richard Grisenthwaite, Sangwon Ha, Alexander Heinecke, Patrick Judd, John Kamalu, et al. Fp8 formats for deep learning. *arXiv preprint arXiv:2209.05433*, 2022.

[38] NVIDIA. CUDA Programming Guide Version 12.4, 2024. URL <https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html>.

[39] Nvidia. Accelerating transformers with nvidia cudnn 9. *Nvidia blog*, 2024. URL <https://developer.nvidia.com/blog/accelerating-transformers-with-nvidia-cudnn-9/>.

[40] NVIDIA. Parallel Thread Execution ISA Version 8.4, 2024. URL https://docs.nvidia.com/cuda/pdf/ptx_isa_8.4.pdf.

[24] Mandy Guo、Joshua Ainslie、David Uthus、Santiago Ontanon、Jianmo Ni、Yun-Hsuan Sung和Yinfei Yang。Long t5：用于长序列的高效文本到文本转换器。arXiv预印本arXiv:2112.07916，2021年。[25] Jonathan Ho、Tim Salimans、Alexey Gritsenko、William Chan、Mohammad Norouzi和David J Fleet。视频扩散模型。神经信息处理系统进展，35：8633–8646，2022年。[26] Coleman Hooper、Sehoon Kim、Hiva Mohammadzadeh、Michael W Mahoney、Yakun Sophia Shao、Kurt Keutzer和Amir Gholami。Kvquant：通过KV缓存量化实现千万上下文长度的LLM推理。arXiv预印本arXiv:2401.18079，2024年。[27] Angelos Katharopoulos、Apoorv Vyas、Nikolaos Pappas和François Fleuret。Transformers是RNN：具有线性注意力的快速自回归Transformers。国际机器学习会议，第5156–5165页。PMLR，2020年。[28] Nikita Kitaev、Łukasz Kaiser和Anselm Levskaya。Reformer：高效的Transformer。国际机器学习会议（ICML），2020年。[29] Woosuk Kwon、Zhuohan Li、Siyuan Zhuang、Ying Sheng、Lianmin Zheng、Cody Hao Yu、Joseph Gonzalez、Hao Zhang和Ion Stoica。基于分页注意力的大语言模型服务高效内存管理。第29届操作系统原理研讨会论文集，第611–626页，2023年。[30] Raymond Li、Loubna Ben Allal、Yangtian Zi、Niklas Muennighoff、Denis Kocetkov、Chenghao Mou、Marc Marone、Christopher Akiki、Jia Li、Jenny Chim等。Starcoder：愿源代码与你同在！arXiv预印本arXiv:2305.06161，2023年。

[31] Hao Liu, Matei Zaharia, and Pieter Abbeel. 用于近无限上下文的块状Transformer环形注意力机制。arXiv preprint arXiv:2310.01889, 2023.

[32] Hao Liu, Wilson Yan, Matei Zaharia, and Pieter Abbeel. 基于环形注意力的百万长度视频与语言世界模型。arXiv preprint arXiv:2402.08268, 2024.

[33] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. Kivi：一种用于KV缓存的免调优非对称2位量化方法。arXiv预印本 arXiv:2402.02750, 2024.

[34] 罗伟乐、范瑞波、李泽宇、杜大有、王强、褚晓文。基准测试与剖析英伟达Hopper GPU架构，2024。URL <https://arxiv.org/abs/2402.13499>。

[35] 马学喆、周春霆、孔翔、何俊贤、桂良轲、Graham Neubig、Jonathan May 和 Luke Zettlemoyer。Mega：配备移动平均的门控注意力机制。收录于《国际学习表征会议》（ICLR），2023年。

[36] 马学哲、杨晓萌、熊文涵、陈贝迪、余丽丽、张浩、Jonathan May、Luke Zettlemoyer、Omer Levy、周春婷。Megalodon：具备无限上下文长度的高效大语言模型预训练与推理。arXiv预印本 arXiv:2404.08801，2024年。

[37] Paulius Micikevicius、Dusan Stolic、Neil Burgess、Marius Cornea、Pradeep Dubey、Richard Grisenthwaite、Sangwon Ha、Alexander Heinecke、Patrick Judd、John Kamalu等。用于深度学习的FP8格式。arXiv预印本arXiv:2209.05433，2022年。

[38] NVIDIA。CUDA编程指南版本12.4，2024年。网址 <https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html>。

[39] Nvidia。使用NVIDIA cuDNN 9加速Transformer模型。Nvidia博客，2024年。URL <https://developer.nvidia.com/blog/accelerating-transformers-with-nvidia-cudnn-9/>。[40] NVIDIA。并行线程执行ISA版本8.4，2024年。URL https://docs.nvidia.com/cuda/pdf/ptx_isa_8.4.pdf。

[41] Muhammad Osama, Duane Merrill, Cris Cecka, Michael Garland, and John D. Owens. Stream-k: Work-centric parallel decomposition for dense matrix-matrix multiplication on the gpu. In *Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*, PPOPP '23, pages 429–431, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9798400700156. doi: 10.1145/3572848.3577479. URL <https://doi.org/10.1145/3572848.3577479>.

[42] Bo Peng, Eric Alcaide, Quentin Anthony, Alon Albalak, Samuel Arcadinho, Huanqi Cao, Xin Cheng, Michael Chung, Matteo Grella, Kranthi Kiran GV, et al. RWKV: Reinventing RNNs for the Transformer era. *arXiv preprint arXiv:2305.13048*, 2023.

[43] Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. Yarn: Efficient context window extension of large language models. *arXiv preprint arXiv:2309.00071*, 2023.

[44] Hao Peng, Nikolaos Pappas, Dani Yogatama, Roy Schwartz, Noah A Smith, and Lingpeng Kong. Random feature attention. In *The International Conference on Learning Representations (ICLR)*, 2021.

[45] Markus N Rabe and Charles Staats. Self-attention does not need $O(n^2)$ memory. *arXiv preprint arXiv:2112.05682*, 2021.

[46] Colfax Research. Tutorial: Matrix Transpose in CUTLASS, 2024. URL <https://research.colfax-intl.com/tutorial-matrix-transpose-in-cutlass/>.

[47] Aurko Roy, Mohammad Saffar, Ashish Vaswani, and David Grangier. Efficient content-based sparse attention with routing Transformers. *arXiv preprint arXiv:2003.05997*, 2020.

[48] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, et al. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.

[49] Rya Sanovar, Srikant Bharadwaj, Renee St. Amant, Victor Rühle, and Saravan Rajmohan. Lean attention: Hardware-aware scalable attention mechanism for the decode-phase of transformers. 2024.

[50] Uri Shaham, Elad Segal, Maor Ivgi, Avia Efrat, Ori Yoran, Adi Haviv, Ankit Gupta, Wenhan Xiong, Mor Geva, Jonathan Berant, et al. Scrolls: Standardized comparison over long language sequences. *arXiv preprint arXiv:2201.03533*, 2022.

[51] Noam Shazeer. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*, 2019.

[52] Benjamin Spector, Aaryan Singhal, Simran Arora, and Christopher Ré, 2024. URL <https://github.com/HazyResearch/ThunderKittens>.

[53] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. Bert4rec: Sequential recommendation with bidirectional encoder representations from transformer. In *Proceedings of the 28th ACM international conference on information and knowledge management*, pages 1441–1450, 2019.

[54] Mingjie Sun, Xinlei Chen, J Zico Kolter, and Zhuang Liu. Massive activations in large language models. *arXiv preprint arXiv:2402.17762*, 2024.

[55] Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. Retentive network: A successor to transformer for large language models. *arXiv preprint arXiv:2307.08621*, 2023.

[56] Yi Tay, Mostafa Dehghani, Dara Bahri, and Donald Metzler. Efficient transformers: A survey. *arXiv preprint arXiv:2009.06732*, 2020.

[57] Vijay Thakkar, Pradeep Ramani, Cris Cecka, Aniket Shivam, Honghao Lu, Ethan Yan, Jack Kosaian, Mark Hoemmen, Haicheng Wu, Andrew Kerr, Matt Nicely, Duane Merrill, Dustyn Blasig, Fengqi Qiao, Piotr Majcher, Paul Springer, Markus Hohnerbach, Jin Wang, and Manish Gupta. CUTLASS, January 2023. URL <https://github.com/NVIDIA/cutlass>.

[41] Muhammad Osama, Duane Merrill, Cris Cecka, Michael Garland 和 John D. Owens。Stream-k：GPU 上稠密矩阵乘法的以工作为中心的并行分解。载于《第 28 届 ACM SIGPLAN 并行编程原理与实践年度研讨会论文集》，PPOPP '23，第 429–431 页，美国纽约州纽约市，2023 年。计算机协会。ISBN 9798400700156. doi: 10.1145/3572848.3577479。URL <https://doi.org/10.1145/3572848.3577479>。[42] Bo Peng、Eric Alcaide、Quentin Anthony、Alon Albalak、Samuel Arcadinho、Huanqi Cao、Xin Cheng、Michael Chung、Matteo Grella、Kranthi Kiran GV 等。RWKV：为 Transformer 时代重塑 RNN。arXiv 预印本 arXiv:2305.13048，2023 年。[43] Bowen Peng、Jeffrey Quesnelle、Honglu Fan 和 Enrico Shippole。Yarn：大语言模型的高效上下文窗口扩展。arXiv 预印本 arXiv:2309.00071，2023 年。[44] Hao Peng、Nikolaos Pappas、Dani Yogatama、Roy Schwartz、Noah A Smith 和 Lingpeng Kong。随机特征注意力。载于《国际学习表征会议 (ICLR) 》，2021 年。[45] Markus N Rabe 和 Charles Staats。自注意力不需要 (n^2) 内存。arXiv 预印本 arXiv:2112.05682，2021 年。[46] Colfax Research。教程：CUTLASS 中的矩阵转置，2024 年。URL <https://research.colfax-intl.com/tutorial-matrix-transpose-in-cutlass/>。

[47] Aurko Roy, Mohammad Saffar, Ashish Vaswani, David Grangier. 基于内容的高效稀疏注意力与路由Transformer。arXiv预印本 arXiv:2003.05997, 2020.

[48] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin 等。Code Llama：用于代码的开放基础模型。arXiv 预印本 arXiv:2308.12950，2023 年。

[49] Rya Sanovar, Srikant Bharadwaj, Renee St. Amant, Victor Rühle 和 Saravan Rajmohan。精益注意力：硬件感知的可扩展注意力机制，用于Transformer的解码阶段。2024年。

[50] Uri Shaham, Elad Segal, Maor Ivgi, Avia Efrat, Ori Yoran, Adi Haviv, Ankit Gupta, Wenhan Xiong, Mor Geva, Jonathan Berant 等。Scrolls：长语言序列的标准化比较。arXiv 预印本 arXiv:2201.03533，2022年。[51] Noam Shazeer。快速Transformer解码：一个写头足矣。arXiv 预印本 arXiv:1911.02150，2019年。[52] Benjamin Spector, Aaryan Singhal, Simran Arora 和 Christopher Ré，2024年。网址 <https://github.com/HazyResearch/ThunderKittens>。[53] 孙飞，刘军，吴健，裴长华，林晓，欧文武，蒋鹏。Bert4rec：基于Transformer双向编码器表示的序列推荐。载于第28届ACM国际信息与知识管理会议论文集，第1441–1450页，2019年。[54] 孙明杰，陈欣蕾，J Zico Kolter，刘壮。大型语言模型中的大规模激活。arXiv 预印本 arXiv:2402.17762，2024年。[55] 孙玉涛，董力，黄少涵，马树铭，夏雨晴，薛继龙，王建勇，韦福如。Retentive网络：Transformer在大型语言模型中的继任者。arXiv 预印本 arXiv:2307.08621，2023年。[56] Yi Tay, Mostafa Dehghani, Dara Bahri, Donald Metzler. 高效Transformer综述。arXiv 预印本 arXiv:2009.06732，2020年。

[57] Vijay Thakkar, Pradeep Ramani, Cris Cecka, Aniket Shivam, Honghao Lu, Ethan Yan, Jack Kosaian, Mark Hoemmen, Haicheng Wu, Andrew Kerr, Matt Nicely, Duane Merrill, Dustyn Blasig, Fengqi Qiao, Piotr Majcher, Paul Springer, Markus Hohnerbach, Jin Wang 和 Manish Gupta。CUTLASS，2023年1月。URL <https://github.com/NVIDIA/cutlass>。

[58] Albert Tseng, Jerry Chee, Qingyao Sun, Volodymyr Kuleshov, and Christopher De Sa. Quip#: Even better llm quantization with hadamard incoherence and lattice codebooks. *arXiv preprint arXiv:2402.04396*, 2024.

[59] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[60] Roger Waleffe, Wonmin Byeon, Duncan Riach, Brandon Norick, Vijay Korthikanti, Tri Dao, Albert Gu, Ali Hatamizadeh, Sudhakar Singh, Deepak Narayanan, et al. An empirical study of mamba-based language models. *arXiv preprint arXiv:2406.07887*, 2024.

[61] Yunyang Xiong, Zhanpeng Zeng, Rudrasis Chakraborty, Mingxing Tan, Glenn Fung, Yin Li, and Vikas Singh. Nyströmformer: A nystöm-based algorithm for approximating self-attention. In *Proceedings of the AAAI Conference on Artificial Intelligence. AAAI Conference on Artificial Intelligence*, volume 35, page 14138, 2021.

[62] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.

[63] Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, et al. Big bird: Transformers for longer sequences. *Advances in Neural Information Processing Systems*, 33, 2020.

[64] Zypfra. Zypfra unveils zamba: A compact 7b ssm hybrid model. *Zypfra blog*, 2024.

[58] Albert Tseng、Jerry Chee、Qingyao Sun、Volodymyr Kuleshov 与 Christopher De Sa。Quip#：利用哈达玛非相干性与格码本实现更优的大语言模型量化。arXiv 预印本 arXiv:2402.04396，2024年。[59] Ashish Vaswani、Noam Shazeer、Niki Parmar、Jakob Uszkoreit、Llion Jones、Aidan N Gomez、Łukasz Kaiser 与 Illia Polosukhin。注意力机制即所需全部。神经信息处理系统进展，第30卷，2017年。[60] Roger Waleffe、Wonmin Byeon、Duncan Riach、Brandon Norick、Vijay Korthikanti、Tri Dao、Albert Gu、Ali Hatamizadeh、Sudhakar Singh、Deepak Narayanan 等。基于 Mamba 架构的语言模型实证研究。arXiv 预印本 arXiv:2406.07887，2024年。[61] Yunyang Xiong、Zhanpeng Zeng、Rudrasis Chakraborty、Mingxing Tan、Glenn Fung、Yin Li 与 Vikas Singh。Nyströmformer：基于 Nyström 方法的自注意力近似算法。收录于人工智能AAAI会议论文集。人工智能AAAI会议，第35卷，第14138页，2021年。[62] Shunyu Yao、Jeffrey Zhao、Dian Yu、Nan Du、Izhak Shafran、Karthik Narasimhan 与 Yuan Cao。React：语言模型中推理与行动的协同机制。arXiv 预印本 arXiv:2210.03629，2022年。[63] Manzil Zaheer、Guru Guruganesh、Kumar Avinava Dubey、Joshua Ainslie、Chris Alberti、Santiago Ontanon、Philip Pham、Anirudh Ravula、Qifan Wang、Li Yang 等。Big Bird：面向长序列的 Transformer 模型。神经信息处理系统进展，第33卷，2020年。

[64] Zypfra。Zypfra 发布 zamba：一款紧凑的 7b ssm 混合模型。Zypfra 博客，2024。

A Related Work

Attention variants and distributed attention Ever since attention became popular with the Transformer architecture [59], there has been a large body of work on approximating attention to scale it to longer sequences. These approximation methods can generally be categorized into two classes: sparse and low-rank. Sparse attention only computes some entries of the attention matrix ($\text{softmax}(\mathbf{Q}\mathbf{K}^T)$) and assumes that other entries are zero. Different methods have different ways of choosing which entries should be zero, either with a fixed pattern [12], with a sliding window [6], or with a dynamic pattern through hashing [28] or routing [47]. The low-rank approach instead assumes that the attention matrix has a low-rank structure, and apply a pointwise nonlinearity to the query and key [27] with random projection [13, 44, 61]. One can also combine the sparse and low-rank approximation for better quality [10, 63]. However, these approximation methods typically do not offer the same model quality as standard attention [56], and so most large-scale models do not employ these techniques.

There are other variants of attention aimed at reducing the size of the KV cache to improve inference efficiency. Multi-query attention [51] and grouped query attention [3] tie different heads of $\mathbf{K}$ and $\mathbf{V}$, and multiple query heads interact with the same key and value head. Multi-head latent attention [19] parameterizes the $\mathbf{K}$ and $\mathbf{V}$ as low-rank projections of a shared matrix to further reduce the KV cache size. However, all of these approaches do not change the core computation $\text{softmax}(\mathbf{Q}\mathbf{K}^T)\mathbf{V}$ during training and simply change how $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ are obtained. As a result, any efficiency or accuracy improvement to the standard attention computation benefits these methods.

To extend to even longer context, attention computation can be distributed across multiple GPUs. Methods such as Ring attention [31, 32] and variants [8] can reach a context length of up to 1 million. They use FLASHATTENTION (or FLASHATTENTION-2) as a primitive, and so the improvement from FLASHATTENTION-3 would benefit these distributed attention methods as well.

Alternative architectures Motivated by the limitations of attention, a variety of alternative architectures have been proposed. They build on the connection between linear attention [27] and recurrent neural networks (RNNs). RWKV [42], H3 [18], MEGA [35], Retnet [55] enhance the expressivity of the simple cumulative sum in linear attention with more sophisticated recurrences. Mamba [22] and xLSTM [5] use learnable weighting for the recurrence and can match the quality of Transformers in language modeling at small or medium scale. These approaches can be connected to generalizations of linear attention through the lens of the structure of the token-mixing matrix [16]. These models have started to see some traction, seeing usage in some medium to large-scale models such as Jamba [2], Zamba [64], Megalodon [36], and Mamba2-hybrid [60]. For the highest quality, these SSM- and RNN-based models still employ many layers of attention. We expect that techniques to speed up attention presented in this work will be useful to speedup these alternative architectures.

Low-precision attention Quantization is a promising approach to speed up attention, but they have mostly focused on reducing the space for KV cache for inference efficiency. QuIP [9] and QuIP#[58] use incoherent processing to reduce the quantization, and we adapted this technique for FP8 FLASHATTENTION-3. Recent work suggests that for inference the KV cache is highly compressible down to 4-, 3-, or even 2-bits [26, 33]. However, quantization during training is still challenging as higher precision is typically required for stable training.

Hardware-aware Algorithms Our work presented in this paper focuses on the micro-architecture specific tuning to leverage new instruction sets and adopt a natively asynchronous programming model. There are other orthogonal axes for hardware-aware algorithm co-design being explored. A recent example of this is LeanAttention [49], which recognizes the poor GPU occupancy and high memory bandwidth requirements of the sequential token generation phase as primary bottlenecks for inference and optimizes it via a smarter load balancing strategy similar to Stream-K load balancing [41] to achieve nearly peak occupancy. There is a large literature on optimizing GEMM for specific hardware that employs many of the same techniques. As an example, Abdelfattah et al. [1] presents a high performance batched GEMM kernel on K40c Graphics Processing Units (GPU) for both fixed and variable sizes, proposing specialized GEMM designs and a comprehensive autotuning process to deliver state-of-the-art performance.

相关工作

注意力变体与分布式注意力 自注意力机制随着ansformer架构[59]的流行而兴起以来，已有大量工作致力于近似注意力计算以扩展到更长的序列。这些近似方法通常可分为两类：稀疏近似与低秩近似。稀疏注意力仅计算注意力矩阵（ $\text{softmax}(\mathbf{Q}\mathbf{K}^T)$ ）中的部分条目，并假设其他条目为零。不同方法通过固定模式[12]、滑动窗口[6]或哈希[28]/路由[47]等动态模式来选择归零条目。低秩方法则假设注意力矩阵具有低秩结构，并通过随机投影[13,44,61]对查询和键施加逐点非线性变换[27]。也可结合稀疏与低秩近似以提升质量[10,63]。然而，这些近似方法通常无法达到标准注意力的模型质量[56]，因此大多数大规模模型并未采用这些技术。

还存在其他旨在减少KV缓存大小以提高推理效率的注意力变体。多查询注意力[51]和分组查询注意力[3]将 $\mathbf{K}$ 和 $\mathbf{V}$ 的不同头进行绑定，使多个查询头与相同的键头和值头交互。多头潜在注意力[19]将 $\mathbf{K}$ 和 $\mathbf{V}$ 参数化为共享矩阵的低秩投影，以进一步减少KV缓存大小。然而，所有这些方法都没有改变训练过程中的核心计算 $\text{softmax}(\mathbf{Q}\mathbf{K}^T)\mathbf{V}$ ，只是改变了 $\mathbf{Q}$ 、 $\mathbf{K}$ 、 $\mathbf{V}$ 的获取方式。因此，任何对标准注意力计算的效率或准确性改进都会使这些方法受益。

为了扩展到更长的上下文，注意力计算可以分布在多个GPU上。诸如环形注意力[31, 32]及其变体[8]等方法可以实现高达100万的上下文长度。它们使用FlashAttention（或FlashAttention-2）作为基础组件，因此FlashAttention-3的改进也将使这些分布式注意力方法受益。

替代架构 受注意力机制局限性的启发，多种替代架构被提出。它们基于线性注意力[27]与循环神经网络（RNN）之间的联系。RWKV[42]、H3[18]、MEGA[35]、Retnet[55]通过更复杂的循环机制增强了线性注意力中简单累加和的表达能力。Mamba[22]和xLSTM[5]使用可学习的权重进行循环计算，能在中小规模语言建模中达到Transformer的质量水平。这些方法可以通过令牌混合矩阵结构的视角[16]与线性注意力的泛化形式联系起来。这些模型已开始获得一定关注，在Jamba[2]、Zamba[64]、Megalodon[36]和Mamba2-hybrid[60]等中大型模型中得到应用。为追求最高质量，这些基于SSM和RNN的模型仍会使用多层注意力机制。我们预计本工作中提出的加速注意力技术将有助于加速这些替代架构。

低精度注意力量化是加速注意力计算的一种有前景的方法，但它们主要集中于减少KV缓存的空间以提高推理效率。QuIP [9] 和 QuIP#[58] 使用非相干处理来降低量化误差，我们将此技术应用于FP8 FlashAttention-3。最近的研究表明，在推理过程中，KV缓存可高度压缩至4位、3位甚至2位 [26, 33]。然而，训练期间的量化仍然具有挑战性，因为稳定训练通常需要更高的精度。

硬件感知算法 本文提出的工作侧重于针对特定微架构的调优，以利用新的指令集并采用原生异步编程模型。目前还有其他正交方向的硬件感知算法协同设计正在探索中。近期的一个例子是LeanAttention [49]，该研究认识到顺序令牌生成阶段GPU占用率低和内存带宽需求高是推理的主要瓶颈，并通过类似Stream-K负载均衡[41]的更智能负载均衡策略进行优化，以实现接近峰值的占用率。针对特定硬件优化GEMM已有大量文献，其中运用了许多相同技术。例如，Abdelfattah等人[1]在K40c图形处理器（GPU）上提出了针对固定和可变尺寸的高性能批量GEMM内核，通过专用GEMM设计和全面的自动调优流程实现了最先进的性能。

B Addition Details on Algorithms

B.1 Asynchrony Through Warp Specialization for the Backward Pass

Similar to the forward pass §3.1, we use warp specialization to handle asynchrony. Instead of just a simple producer-consumer pattern in the forward pass, we add one extra role of a **dQ** writer, since we need to accumulate the value of **dQ** produced by each thread block to the global value of **dQ**. This **dQ** accumulation introduces memory contention (many thread blocks writing to the same location) so having a separate warp to handle this (along with asynchrony) will avoid blocking the rest of the warps in the thread block to perform the next computation (matmul).

We include the backward pass with warp specialization in Algorithm 3.

Algorithm 3 FLASHATTENTION-3 backward pass with warp specialization

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}, \mathbf{dO} \in \mathbb{R}^{N \times d}$ in HBM, logsumexp vector $L \in \mathbb{R}^N$ in HBM, block sizes B_c, B_r .

- 1: In a preprocessing kernel, compute $D = \text{rowsum}(\mathbf{dO} \circ \mathbf{O}) \in \mathbb{R}^d$ (pointwise multiply), write D to HBM and divide it into T_r blocks $D_1, \dots, D_{T_r}$ of size B_r each.
- 2: Divide $\mathbf{Q}$ into $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
- 3: Divide $\mathbf{dO}$ into T_r blocks $\mathbf{dO}_1, \dots, \mathbf{dO}_{T_r}$ of size $B_r \times d$ each, and divide L into T_r blocks $L_1, \dots, L_{T_r}$ of size B_r each.
- 4: Initialize pipeline object to manage barrier synchronization with s -stage circular SMEM buffer.
- 5: **if** in producer warpgroup **then**
- 6: Deallocate predetermined number of registers.
- 7: Issue load $\mathbf{K}_j$ and $\mathbf{V}_j$ from HBM to shared memory.
- 8: Upon completion, commit to notify consumer of the load of $\mathbf{K}_j$ and $\mathbf{V}_j$.
- 9: **for** $1 \leq i \leq T_r$ **do**
- 10: Wait for the $(i \% s)$ th stage of the buffer to be consumed.
- 11: Issue loads of $\mathbf{Q}_i, \mathbf{dO}_i$ from HBM to shared memory at the $(i \% s)$ th stage of the buffer.
- 12: Upon completion, commit to notify consumers of the loads of $\mathbf{Q}_i, \mathbf{dO}_i$.
- 13: **end for**
- 14: **else if** in consumer warpgroups **then**
- 15: Reallocate predetermined number of registers as function of number of consumer warps.
- 16: On-chip, Initialize $\mathbf{dK}_j = (0)_{B_c \times d}, \mathbf{dV}_j = (0)_{B_c \times d}$.
- 17: Wait for $\mathbf{K}_j$ and $\mathbf{V}_j$ to be loaded in shared memory.
- 18: **for** $1 \leq i \leq T_r$ **do**
- 19: Wait for $\mathbf{Q}_i$ to be loaded in shared memory.
- 20: Load L_i, D_i from HBM to on-chip SRAM.
- 21: On chip, compute $\mathbf{S}_i^{(j)} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$ (SS-GEMM). Commit.
- 22: Wait for $\mathbf{dO}_i$ to be loaded in shared memory.
- 23: On chip, compute $\mathbf{dP}_i^{(j)} = \mathbf{dO}_i \mathbf{V}_j^T \in \mathbb{R}^{B_r \times B_c}$ (SS-GEMM). Commit.
- 24: On chip, wait for $\mathbf{S}_i^{(j)}$, then compute $\mathbf{P}_i^{(j)} = \exp(\mathbf{S}_{ij} - L_i) \in \mathbb{R}^{B_r \times B_c}$.
- 25: On chip, wait for $\mathbf{dP}_i^{(j)}$, then compute $\mathbf{dS}_i^{(j)} = \mathbf{P}_i^{(j)} \circ (\mathbf{dP}_i^{(j)} - D_i) \in \mathbb{R}^{B_r \times B_c}$.
- 26: On chip, compute $\mathbf{dV}_j \leftarrow \mathbf{dV}_j + (\mathbf{P}_i^{(j)})^T \mathbf{dO}_i \in \mathbb{R}^{B_c \times d}$ (RS-GEMM). Commit.
- 27: On chip, compute $\mathbf{dK}_j \leftarrow \mathbf{dK}_j + \mathbf{dS}_i^{(j)T} \mathbf{Q}_i \in \mathbb{R}^{B_c \times d}$ (RS-GEMM). Commit and wait for both $\mathbf{dV}_j$ and $\mathbf{dK}_j$.
- 28: On chip, compute $\mathbf{dQ}_i^{(\text{local})} = \mathbf{dS}_i^{(j)} \mathbf{K}_j \in \mathbb{R}^{B_r \times d}$ (SS-GEMM), and write $\mathbf{dQ}_i^{(\text{local})}$ to smem. Notify the **dQ**-writer.
- 29: **end for**
- 30: **else if** in **dQ**-writer warp **then**
- 31: **for** $1 \leq i \leq T_r$ **do**
- 32: Wait for $\mathbf{dQ}_i^{(\text{local})}$ to be ready in smem.
- 33: Using a semaphore, atomically add $\mathbf{dQ}_i^{(\text{local})}$ to $\mathbf{dQ}_i$ in global memory.
- 34: **end for**
- 35: **end if**

B 算法补充细节

B.1 通过Warp专业化实现反向传播的异步处理

与正向传播§3.1类似，我们使用warp specialization来处理异步性。不同于正向传播中简单的生产者-消费者模式，我们额外增加了一个**dQ**写入器的角色，因为需要将每个线程块生成的**dQ**值累加到**dQ**的全局值中。这种**dQ**累加操作会引发内存争用（多个线程块向同一位置写入），因此使用独立的warp来处理此操作（同时处理异步性）将避免线程块中其余warp因等待而阻塞，从而能继续执行后续计算（矩阵乘法）。

我们在算法3中加入了包含warp specialization的后向传播。

Algorithm 3 FLASHATTENTION-3 backward pass with warp specialization

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}, \mathbf{dO} \in \mathbb{R}^{N \times d}$ in HBM, logsumexp vector $L \in \mathbb{R}^N$ in HBM, block sizes B_c, B_r .

- 1: In a preprocessing kernel, compute $D = \text{rowsum}(\mathbf{dO} \circ \mathbf{O}) \in \mathbb{R}^d$ (pointwise multiply), write D to HBM and divide it into T_r blocks $D_1, \dots, D_{T_r}$ of size B_r each.
- 2: Divide $\mathbf{Q}$ into $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
- 3: Divide $\mathbf{dO}$ into T_r blocks $\mathbf{dO}_1, \dots, \mathbf{dO}_{T_r}$ of size $B_r \times d$ each, and divide L into T_r blocks $L_1, \dots, L_{T_r}$ of size B_r each.
- 4: Initialize pipeline object to manage barrier synchronization with s -stage circular SMEM buffer.
- 5: **if** in producer warpgroup **then**
- 6: Deallocate predetermined number of registers.
- 7: Issue load $\mathbf{K}_j$ and $\mathbf{V}_j$ from HBM to shared memory.
- 8: Upon completion, commit to notify consumer of the load of $\mathbf{K}_j$ and $\mathbf{V}_j$.
- 9: **for** $1 \leq i \leq T_r$ **do**
- 10: Wait for the $(i \% s)$ th stage of the buffer to be consumed.
- 11: Issue loads of $\mathbf{Q}_i, \mathbf{dO}_i$ from HBM to shared memory at the $(i \% s)$ th stage of the buffer.
- 12: Upon completion, commit to notify consumers of the loads of $\mathbf{Q}_i, \mathbf{dO}_i$.
- 13: **end for**
- 14: **else if** in consumer warpgroups **then**
- 15: Reallocate predetermined number of registers as function of number of consumer warps.
- 16: On-chip, Initialize $\mathbf{dK}_j = (0)_{B_c \times d}, \mathbf{dV}_j = (0)_{B_c \times d}$.
- 17: Wait for $\mathbf{K}_j$ and $\mathbf{V}_j$ to be loaded in shared memory.
- 18: **for** $1 \leq i \leq T_r$ **do**
- 19: Wait for $\mathbf{Q}_i$ to be loaded in shared memory.
- 20: Load L_i, D_i from HBM to on-chip SRAM.
- 21: On chip, compute $\mathbf{S}_i^{(j)} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$ (SS-GEMM). Commit.
- 22: Wait for $\mathbf{dO}_i$ to be loaded in shared memory.
- 23: On chip, compute $\mathbf{dP}_i^{(j)} = \mathbf{dO}_i \mathbf{V}_j^T \in \mathbb{R}^{B_r \times B_c}$ (SS-GEMM). Commit.
- 24: On chip, wait for $\mathbf{S}_i^{(j)}$, then compute $\mathbf{P}_i^{(j)} = \exp(\mathbf{S}_{ij} - L_i) \in \mathbb{R}^{B_r \times B_c}$.
- 25: On chip, wait for $\mathbf{dP}_i^{(j)}$, then compute $\mathbf{dS}_i^{(j)} = \mathbf{P}_i^{(j)} \circ (\mathbf{dP}_i^{(j)} - D_i) \in \mathbb{R}^{B_r \times B_c}$.
- 26: On chip, compute $\mathbf{dV}_j \leftarrow \mathbf{dV}_j + (\mathbf{P}_i^{(j)})^T \mathbf{dO}_i \in \mathbb{R}^{B_c \times d}$ (RS-GEMM). Commit.
- 27: On chip, compute $\mathbf{dK}_j \leftarrow \mathbf{dK}_j + \mathbf{dS}_i^{(j)T} \mathbf{Q}_i \in \mathbb{R}^{B_c \times d}$ (RS-GEMM). Commit and wait for both $\mathbf{dV}_j$ and $\mathbf{dK}_j$.
- 28: On chip, compute $\mathbf{dQ}_i^{(\text{local})} = \mathbf{dS}_i^{(j)} \mathbf{K}_j \in \mathbb{R}^{B_r \times d}$ (SS-GEMM), and write $\mathbf{dQ}_i^{(\text{local})}$ to smem. Notify the **dQ**-writer.
- 29: **end for**
- 30: **else if** in **dQ**-writer warp **then**
- 31: **for** $1 \leq i \leq T_r$ **do**
- 32: Wait for $\mathbf{dQ}_i^{(\text{local})}$ to be ready in smem.
- 33: Using a semaphore, atomically add $\mathbf{dQ}_i^{(\text{local})}$ to $\mathbf{dQ}_i$ in global memory.
- 34: **end for**
- 35: **end if**

B.2 2-Stage Pipelining SASS Analysis

We give simplified SASS code for the inside of the consumer warpgroup mainloop.

```
// Compute row_max
FMNMX.FTZ R0, R24, R6, !PT ;
SHFL.BFLY PT, R185, R2, 0x2, 0x1f ;
... FMNMX and SHFL.BFLY ...

// Apply exp2 and row_sum. Rescale 0.
FMUL.FTZ R2, R4, UR9 ;
MUFU.EX2 R185, R184 ;
FFMA.FTZ R24, R24, UR9, -R6.reuse ;
FADD.FTZ R24, R211, R24 ;
... FMUL, FFMA, FMUL, MUFU.EX2, FADD ...

// FP32 -> FP16 conversion are interleaved with exp2, row_sum and 0 rescaling.
F2FP.F16.F32.PACK_AB R231, R25, R231 ;
... F2FP, FMUL, MUFU, FFMA, FADD ...

// Start the first WGMMA. Broken down into 8 HGMMAs.
// The first 7 HGMMAs are packed together.
WARPGROUP.ARRIVE ;
HGMMMA.64x192x16.F32 R24, gdesc[UR44], RZ, !UPT ;
... HGMMMA x 6 ...

// FP32->FP16, exp2, row_sum, 0 rescaling are interleaved with HGMMMA.
F2FP.F16.F32.PACK_AB R214, R214, R187 ;
MUFU.EX2 R234, R5 ;
FADD.FTZ R237, R187, R2 ;
... F2FP, MUFU, FADD ...

// The last HGMMMA is issued here. No need to wait.
HGMMMA.64x192x16.F32 R24, gdesc[UR44], R24, gsb0 ;

// Start the second WGMMA. Broken down into 12 HGMMAs.
// All 12 HGMMAs are packed together. Not interleaved with other instructions.
WARPGROUP.ARRIVE ;
HGMMMA.64x128x16.F32 R120, R228, gdesc[UR8].tnspB, R120 ;
... HGMMMA x 10 ...
HGMMMA.64x128x16.F32 R120, R184, gdesc[UR8].tnspB, R120, gsb0 ;

// wgmma.wait_group at the end.
WARPGROUP.DEPBAR.LE gsb0, 0x0 ;
```

- We make the following observations:
1. Softmax is reordered to the very beginning, even before the first WGMMA.
 2. The first WGMMA is interleaved with softmax and FP32 → FP16 datatype conversion of **S**. This indicates that WGMMA and non-WGMMAs are executed in parallel.
 3. `exp2`, `row_sum`, `O` rescaling and FP32 → FP16 conversions are interleaved together.
 4. The second WGMMA is not overlapped with other instructions, as expected.
- Overall, SASS shows that the 2-stage pipelining idea works as expected.

B.2 两级流水线SASS分析

我们给出简化 消费者warpgroup内部的SASS代码 主循环。

```
// 计算行最大值 FMNMX.FTZ R0, R24, R6,
!PT ; SHFL.BFLY PT, R185, R2, 0x2, 0x1f ;
... FMNMX 和 SHFL.BFLY ...

// 应用exp2和行求和。重新缩放O。 FMUL.FTZ
R2, R4, UR9 ; MUFU.EX2 R185, R184 ; FFMA.FT
Z R24, R24, UR9, -R6.reuse ; FADD.FTZ R24, R21
1, R24 ; ... FMUL, FFMA, FMUL, MUFU.EX2, FA
DD ...

// FP32 -> FP16 转换与 exp2、row_sum 和 O 重新缩放交错进行。 F2FP.F16.F32.PACK_AB R231
, R25, R231 ; ... F2FP, FMUL, MUFU, FFMA, FADD ...

// 启动第一个WGMMA。分解为8个HGMMMA。// 前7个HGMM
A打包在一起。WARPGROUP.ARRIVE ;HGMMMA.64x192x16.F
32 R24, gdesc[UR44], RZ, !UPT ;... HGMMMA x 6 ...

// FP32->FP16、exp2、行求和、O重缩放与HGMMMA交错进行。 F2FP.F16.F32.PACK
_AB R214, R214, R187 ; MUFU.EX2 R234, R5 ; FADD.FTZ R237, R187, R2 ; ... F2FP
、MUFU、FADD ...

// 最后一个HGMMMA在此处发出。无需等待。 HGMMMA.64x1
92x16.F32 R24, gdesc[UR44], R24, gsb0 ;

// 启动第二个WGMMA。分解为12个HGMMMA。// 所有12个HGMMMA打包在一起。不与其他
指令交错 。
WARPGROUP.ARRIVE ; HGMMMA.64x128x16.F32 R120, R228, gdesc[UR8].t
nspB, R120 ; ... HGMMMA x 10 ... HGMMMA.64x128x16.F32 R120, R184, gdesc[
UR8].tnspB, R120, gsb0 ;

// 在末尾使用wgmma.wait_group。 W
ARPGROUP.DEPBAR.LE gsb0, 0x0 ;
```

- 我们做出以下观察：
1. Softmax被重新排序到最前面，甚至在第一个WGMMA之前。2. 第一个WGMMA与softmax以及S的FP32 → FP16数据类型转换交错执行。这表明WGMMA和非WGMMA是并行执行的。
 3. `exp2`、行求和、O重新缩放以及FP32 → FP16转换交错进行。
 4. 第二个WGMMA指令如预期那样，未与其他指令重叠。
- 总体而言，SASS表明，两阶段流水线设计的思路如预期般奏效。

B.3 3-Stage Pipelining Algorithm

We experiment with a 3-stage pipelining algorithm to parallelize the first WGMMA from iteration $j + 2$, softmax from iteration $j + 1$, and the second WGMMA from iteration j . We describe this algorithm in Algorithm 4. This algorithm behaves worse than the 2-stage pipelining algorithm due to the reasons below:

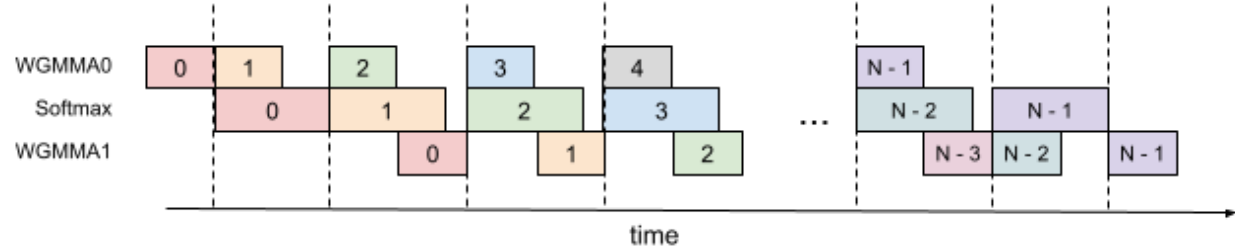


Figure 8: 3-Stage Pipelining

Algorithm 4 FLASHATTENTION 3-stage pipelining consumer warpgroup forward pass

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, block sizes B_c, B_r . Each warpgroup reads 1 block $\mathbf{Q}_i$ of size $B_r \times d$, $T_c = \lceil \frac{N}{B_c} \rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$ of size $B_c \times d$. Each warpgroup writes 1 output block $\mathbf{O}_i$ of size $B_r \times d$, and 1 logsumexp block L_i of size B_r .

- 1: Initialization. Load $\mathbf{Q}_i$ from HBM to on-chip SRAM. Initialize $\mathbf{O}_i, \ell_i, m_i, scale_o$.
- 2: Wait for the producer warpgroup loading $\mathbf{K}_0$ from HBM to on-chip SRAM.
- 3: Compute $\mathbf{S} = \mathbf{Q}_i \mathbf{K}_0^T$ using WGMMA. Commit and wait.
- 4: Compute $m_i, \tilde{\mathbf{P}}_i, \ell_i, scale_o$ based on $\mathbf{S}$.
- 5: Wait for the producer warpgroup loading $\mathbf{K}_1$ from HBM to on-chip SRAM.
- 6: Compute $\mathbf{S} = \mathbf{Q}_i \mathbf{K}_1^T$ using WGMMA. Commit and wait.
- 7: **for** $2 \leq j < T_c - 2$ **do**
- 8: Wait for the producer warpgroup loading $\mathbf{K}_j$ from HBM to on-chip SRAM.
- 9: Compute $\mathbf{S}_{next} = \mathbf{Q}_i \mathbf{K}_j^T$ using WGMMA. Commit but do not wait.
- 10: Wait for the producer warpgroup loading $\mathbf{V}_{j-2}$ from HBM to on-chip SRAM.
- 11: Rescale $\mathbf{O}_i$ based on $scale_o$.
- 12: Compute $\mathbf{O}_i = \mathbf{O}_i + \tilde{\mathbf{P}}_i \mathbf{V}_{j-2}$ using WGMMA. Commit but do not wait.
- 13: Compute $m_i, \tilde{\mathbf{P}}_{i,next}, \ell_i, scale_o$ based on $\mathbf{S}$.
- 14: Wait for all previous WGMMA.
- 15: Copy $\mathbf{S}_{next}$ to $\mathbf{S}$.
- 16: Copy $\tilde{\mathbf{P}}_{i,next}$ to $\tilde{\mathbf{P}}_i$.
- 17: **end for**
- 18: Wait for the producer warpgroup loading $\mathbf{V}_{T_c-2}$ from HBM to on-chip SRAM.
- 19: Rescale $\mathbf{O}_i$ based on $scale_o$.
- 20: Compute $\mathbf{O}_i = \mathbf{O}_i + \tilde{\mathbf{P}}_i \mathbf{V}_{T_c-2}$ using WGMMA. Commit and wait.
- 21: Compute $m_i, \tilde{\mathbf{P}}_i, \ell_i, scale_o$ based on $\mathbf{S}$.
- 22: Wait for the producer warpgroup loading $\mathbf{V}_{T_c-1}$ from HBM to on-chip SRAM.
- 23: Rescale $\mathbf{O}_i$ based on $scale_o$.
- 24: Compute $\mathbf{O}_i = \mathbf{O}_i + \tilde{\mathbf{P}}_i \mathbf{V}_{T_c-1}$ using WGMMA. Commit and wait.
- 25: Epilogue. Rescale $\mathbf{O}_i$ based on ℓ_i . Compute L_i based on ℓ_i and m_i . Write $\mathbf{O}_i$ and L_i to HBM as the i -th block of $\mathbf{O}$ and L .

Overlapping. We expected that softmax can be overlapped with (the first WGMMA + the second WGMMA). However, the compiler doesn't cooperate in this way. SASS code shows that only the first WGMMA is overlapped with softmax, while the second WGMMA is not. It's not clear why the compiler chooses to reorder instructions in this way.

B.3 三级流水线算法

我们尝试采用一种三阶段流水线算法，将第一次WGMMA从迭代 + 2、softmax从迭代 + 1以及第二次WGMMA从迭代 并行化。我们在算法中描述了这一算法。由于以下原因，该算法的表现不如两阶段流水线算法：

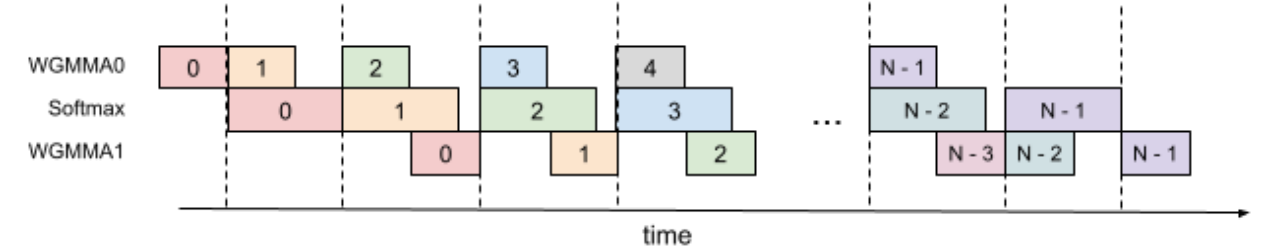


图8: 3级流水线

Algorithm 4 FLASHATTENTION 3-stage pipelining consumer warpgroup forward pass

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, block sizes B_c, B_r . Each warpgroup reads 1 block $\mathbf{Q}_i$ of size $B_r \times d$, $T_c = \lceil \frac{N}{B_c} \rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$ of size $B_c \times d$. Each warpgroup writes 1 output block $\mathbf{O}_i$ of size $B_r \times d$, and 1 logsumexp block L_i of size B_r .

- 1: Initialization. Load $\mathbf{Q}_i$ from HBM to on-chip SRAM. Initialize $\mathbf{O}_i, \ell_i, m_i, scale_o$.
- 2: Wait for the producer warpgroup loading $\mathbf{K}_0$ from HBM to on-chip SRAM.
- 3: Compute $\mathbf{S} = \mathbf{Q}_i \mathbf{K}_0^T$ using WGMMA. Commit and wait.
- 4: Compute $m_i, \tilde{\mathbf{P}}_i, \ell_i, scale_o$ based on $\mathbf{S}$.
- 5: Wait for the producer warpgroup loading $\mathbf{K}_1$ from HBM to on-chip SRAM.
- 6: Compute $\mathbf{S} = \mathbf{Q}_i \mathbf{K}_1^T$ using WGMMA. Commit and wait.
- 7: **for** $2 \leq j < T_c - 2$ **do**
- 8: Wait for the producer warpgroup loading $\mathbf{K}_j$ from HBM to on-chip SRAM.
- 9: Compute $\mathbf{S}_{next} = \mathbf{Q}_i \mathbf{K}_j^T$ using WGMMA. Commit but do not wait.
- 10: Wait for the producer warpgroup loading $\mathbf{V}_{j-2}$ from HBM to on-chip SRAM.
- 11: Rescale $\mathbf{O}_i$ based on $scale_o$.
- 12: Compute $\mathbf{O}_i = \mathbf{O}_i + \tilde{\mathbf{P}}_i \mathbf{V}_{j-2}$ using WGMMA. Commit but do not wait.
- 13: Compute $m_i, \tilde{\mathbf{P}}_{i,next}, \ell_i, scale_o$ based on $\mathbf{S}$.
- 14: Wait for all previous WGMMA.
- 15: Copy $\mathbf{S}_{next}$ to $\mathbf{S}$.
- 16: Copy $\tilde{\mathbf{P}}_{i,next}$ to $\tilde{\mathbf{P}}_i$.
- 17: **end for**
- 18: Wait for the producer warpgroup loading $\mathbf{V}_{T_c-2}$ from HBM to on-chip SRAM.
- 19: Rescale $\mathbf{O}_i$ based on $scale_o$.
- 20: Compute $\mathbf{O}_i = \mathbf{O}_i + \tilde{\mathbf{P}}_i \mathbf{V}_{T_c-2}$ using WGMMA. Commit and wait.
- 21: Compute $m_i, \tilde{\mathbf{P}}_i, \ell_i, scale_o$ based on $\mathbf{S}$.
- 22: Wait for the producer warpgroup loading $\mathbf{V}_{T_c-1}$ from HBM to on-chip SRAM.
- 23: Rescale $\mathbf{O}_i$ based on $scale_o$.
- 24: Compute $\mathbf{O}_i = \mathbf{O}_i + \tilde{\mathbf{P}}_i \mathbf{V}_{T_c-1}$ using WGMMA. Commit and wait.
- 25: Epilogue. Rescale $\mathbf{O}_i$ based on ℓ_i . Compute L_i based on ℓ_i and m_i . Write $\mathbf{O}_i$ and L_i to HBM as the i -th block of $\mathbf{O}$ and L .

重叠。我们预期softmax可以与（第一个WGMMA + 第二个WGMMA）重叠执行。然而，编译器并未按此方式协作。SASS代码显示，仅第一个WGMMA与softmax重叠，而第二个WGMMA则未重叠。目前尚不清楚编译器为何选择以这种方式重排指令。

Register pressure. This algorithm requires more registers compared to the 2-stage pipelining algorithm. In theory, it needs to store an extra $\tilde{\mathbf{P}}_i$ and $scale_o$, which is of size $B_r \times B_c \times \text{sizeof}(\text{input_data_type}) + B_r \times \text{sizeof}(\text{float})$. As a result, a smaller block size needs to be chosen.

C Addition Details on Experiments and Benchmarking

C.1 System and libraries

We benchmark the speed on an H100 80GB SXM5 (700W). We generally use the latest versions of the libraries, at the time of writing (May 2024). Specifically, we use:

- CUDA 12.3
- cuDNN 9.1.1.17
- CUTLASS 3.5
- FLASHATTENTION 2.5.8
- Triton nightly 3.0.0.post20240424212437
- PyTorch 2.3.0

To reduce variability, we fix the GPU clock speed to 1830MHz (clock speed used to calculate the 989 TFLOPS FP16 theoretical max throughput). We repeat the benchmarks 100 times and take the average timing.

C.2 FP8 Attention Full Results

We use following sequence lengths: 512, 1024, 2048, 4224, 8448, 16896. When sequence length $\geq 4\text{k}$, we make it also divisible by 132 (number of SMs in H100 SXM5) to avoid wave quantization.

寄存器压力。与两阶段流水线算法相比，该算法需要更多的寄存器。理论上，它需要额外存储一个 $\tilde{\mathbf{P}}_i$ 和 $scale_o$ ，其大小为 $B_r \times B_c \times \text{sizeof}(\text{input_data_type}) + B_r \times \text{sizeof}(\text{float})$ 。因此，需要选择更小的块大小。

C 实验与基准测试的补充详情

C.1 系统与库

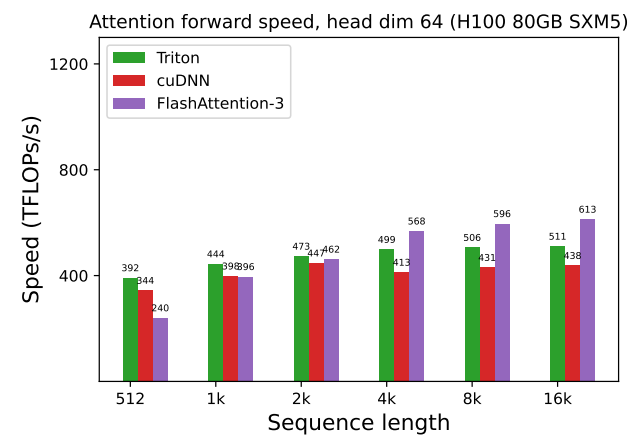
我们在H100 80GB SXM5（700W）上对速度进行了基准测试。我们通常使用撰写本文时（2024年5月）最新的库版本。具体来说，我们使用：

- CUDA 12.3
- cuDNN 9.1.1.17
- CUTLASS 3.5
- FlashAttention 2.5.8
- Triton nightly 版本 3.0.0.post20240424212437
- PyTorch 2.3.0

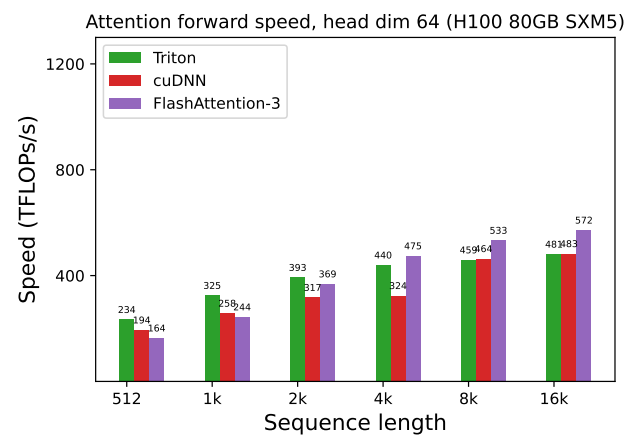
为了减少波动性，我们将GPU时钟速度固定为1830MHz（该时钟速度用于计算989 TFLOPS FP16的理论最大吞吐量）。我们将基准测试重复100次并取平均时间。

C.2 FP8注意力完整结果

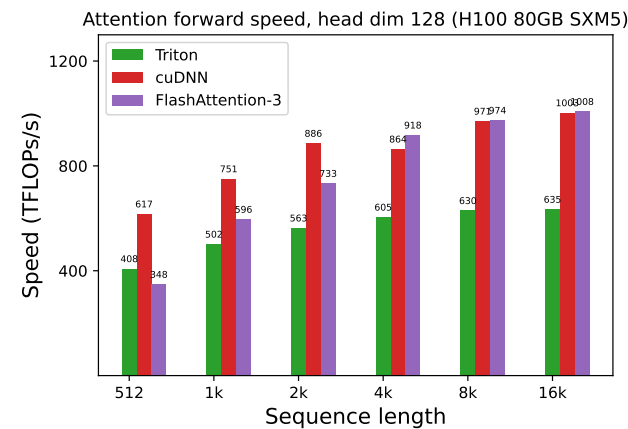
我们使用以下序列长度：512、1024、2048、4224、8448、16896。当序列长度 $\geq 4\text{k}$ 时，我们使其同样可被132（H100 SXM5中的SM数量）整除，以避免波量化。



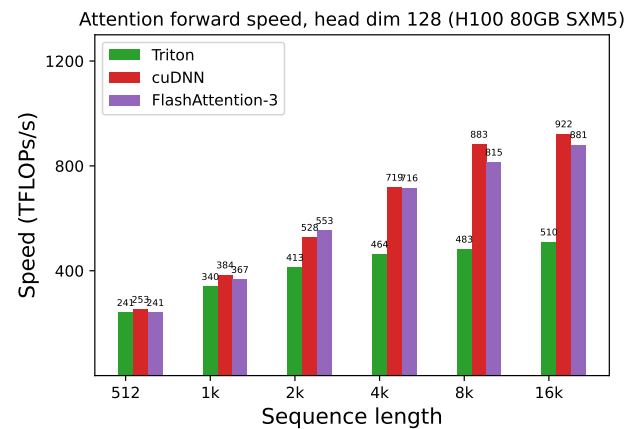
(a) Forward, without causal mask, head dim 64



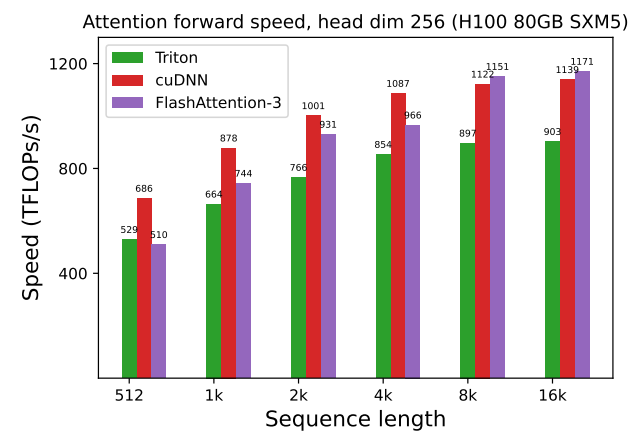
(b) Forward, with causal mask, head dim 64



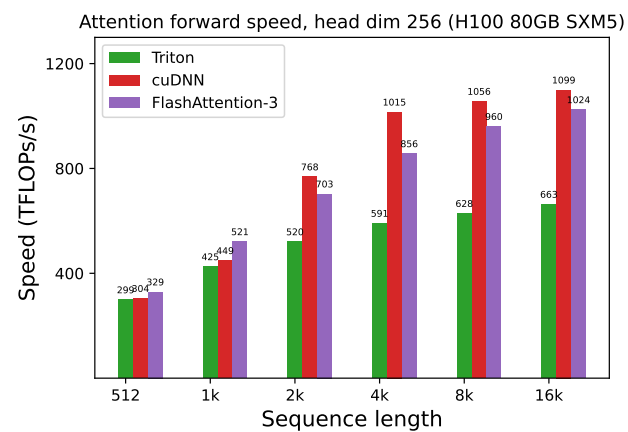
(c) Forward, without causal mask, head dim 128



(d) Forward, with causal mask, head dim 128

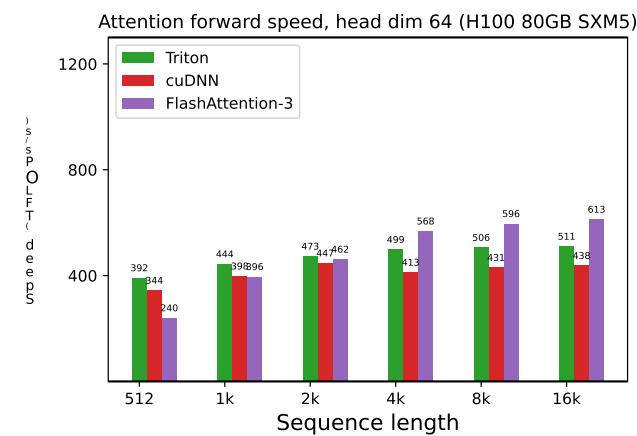


(e) Forward, without causal mask, head dim 256

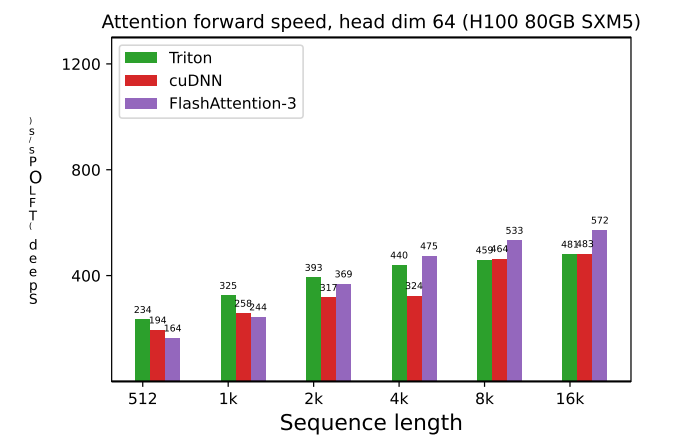


(f) Forward, with causal mask, head dim 256

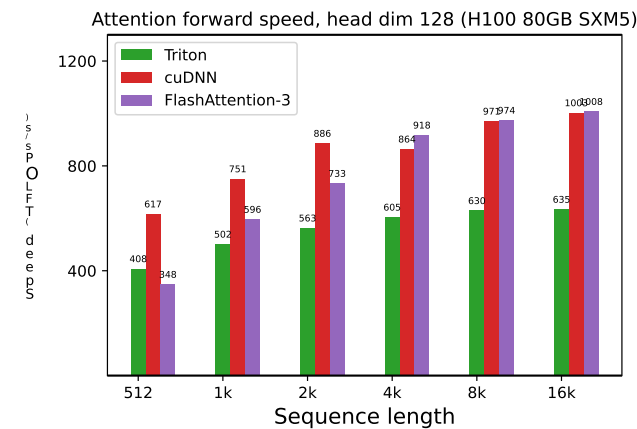
Figure 9: Attention forward speed (FP8) on H100 GPU



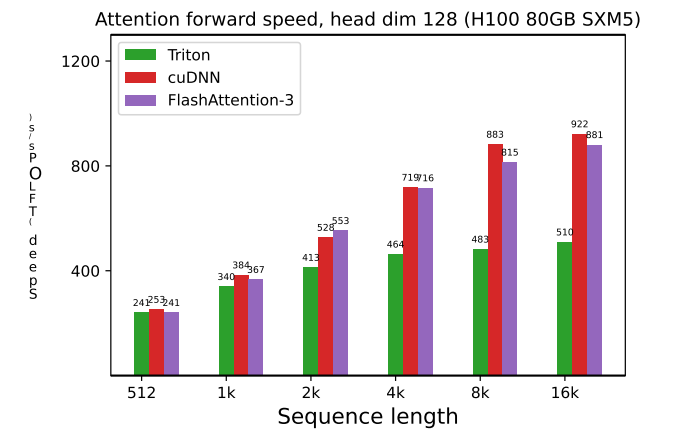
(a) Forward, without causal mask, head dim 64



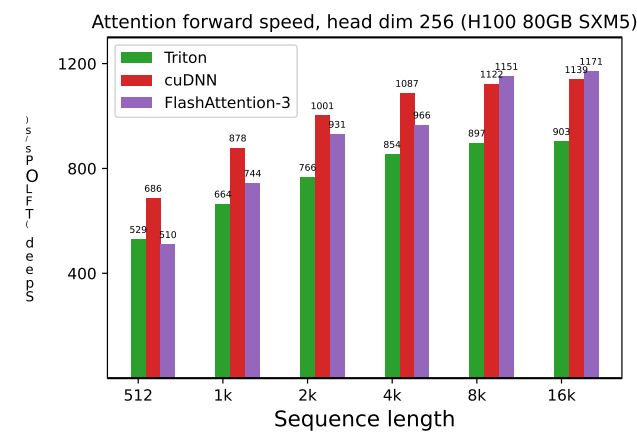
(b) Forward, with causal mask, head dim 64



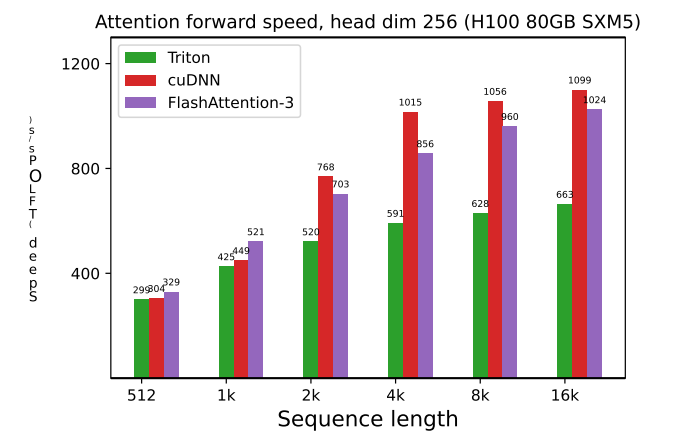
(c) Forward, without causal mask, head dim 128



(d) Forward, with causal mask, head dim 128



(e) Forward, without causal mask, head dim 256



(f) Forward, with causal mask, head dim 256

图9: H100 GPU上的注意力前向速度 (FP8)